

Formation Timescale

Ali Koudri - ali.koudri@janus-academy.fr



Ali Koudri

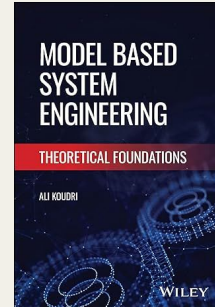
- PhD en informatique
- 25+ ans en ingénierie système et logicielle, modélisation, performance, sécurité
- IRT SystemX - Thales - CEA
- Janus Academy : formation & conseil indépendant
- Approche cross-stack : front, back, ci/cd, observabilité

Publications

Model Based Systems Engineering: Theoretical Foundations

Posture pédagogique

Démarche systémique, mesure avant optimisation, capitalisation transposable



Agenda

- Problèmes, motivations et concepts
- Déploiement et mise en service
- Modélisation des données temporelles
- Chunks, index et exclusion
- Ingestion haut débit
- Requêtes temporelles
- Hyperfonctions et agréabilité
- Agrégats continus
- Hypercore et columnstore
- Cycle de vie de la donnée
- Automatisation et jobs
- Migration
- Sauvegarde, restauration et versions
- Sécurité et conformité
- Diagnostic et supervision



MODULE M01

Problèmes, motivations, concepts

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Agenda

A Amorce · deux requêtes, une seule différence

1 Les six pathologies

2 Étendre PostgreSQL : quand oui, quand non

3 Les douze concepts

4 Nomenclature, éditions, versions

✓ Point d'acquis · carte des concepts

Objectifs

1

Nommer le problème

Rattacher un symptôme observé en production à un concept du produit, plutôt qu'à une intuition ou à un nom de fonction.

2

Décider, y compris de dire non

Déterminer si TimescaleDB est une réponse pertinente pour une charge donnée, et savoir énoncer les cas où elle ne l'est pas.

Ce module ne comporte pas d'atelier. Il installe le vocabulaire utilisé sur les quatorze modules suivants.

Deux requêtes, une seule différence

La même agrégation horaire, exécutée sur la base MISTRAL d'aujourd'hui, puis sur la cible construite au fil des cinq jours.

```
SELECT  time_bucket('1 hour', ts) AS heure,  
        avg(valeur)  
FROM    mesures  
WHERE   ts >= (SELECT max(ts) FROM mesures) - interval '30 days'  
GROUP BY heure;
```

mistral_legacy

41,3 s

PostgreSQL 16, table partitionnée à la main

Cible du module M10

0,18 s

Hypertable, agrégat continu, columnstore

L'écart n'est pas commenté maintenant. Il est affiché, et sert de dette à rembourser pendant cinq jours.



BLOC 1.1

Les six pathologies

Six symptômes observables. Chacun est rejoué en atelier plus loin dans la formation.

Ce que l'on observe avant de le nommer

1

Le débit d'insertion s'effondre

Traité en M04 · M05

2

La purge bloque la production

Traité en M10

3

Les mêmes agrégations sont recalculées

Traité en M08

4

Les données de 2023 coûtent le prix d'hier

Traité en M09 · M10

5

Les moyennes sont fausses

Traité en M07

6

Référentiel et mesures vivent séparés

Traité en M03 · M12

Écriture et suppression

1

Le débit d'insertion s'effondre au-delà de quelques centaines de millions de lignes

Cause structurelle Index B-tree qui ne tient plus en mémoire, écritures dispersées sur des pages froides

Traité en **M04 (dimensionnement)** · **M05 (ingestion)**

2

La purge des données anciennes bloque la production une nuit par mois

Cause structurelle DELETE massif, bloat consécutif, autovacuum durablement en retard

Traité en **M10 (cycle de vie)**

Lecture et coût de stockage

3

Les mêmes agrégations sont recalculées à chaque affichage du tableau de bord

Cause structurelle Aucune matérialisation incrémentale : la vue matérialisée classique recalcule tout

Traité en **M08 (agrégats continus)**

4

Les données de 2023 coûtent exactement le même prix que celles d'hier

Cause structurelle Aucune séparation entre données chaudes, tièdes et froides

Traité en **M09 (columnstore) · M10 (rétention)**

Justesse et modélisation

5

Les moyennes sont fausses sur les capteurs à pas d'échantillonnage irrégulier

Cause structurelle Agrégation non pondérée par le temps : chaque point compte pareil quelle que soit sa durée

Traité en **M07 (agrégabilité)**

6

Le référentiel métier et les mesures vivent dans deux systèmes distincts

Cause structurelle Silo temporel : la jointure entre une série et sa machine devient un travail applicatif

Traité en **M03 (modélisation) · M12 (migration)**

MISTRAL aujourd'hui

42 éoliennes, 3 centrales photovoltaïques, la métrologie du point de livraison réseau. Une seule base PostgreSQL 16.

200 Go

volume actuel

40 s

ouverture d'un tableau de bord

1 nuit / mois

production bloquée par la purge

Ce que la base contient déjà

- Une table de mesures partitionnée à la main, dont les partitions sont créées par un script
- Des vues matérialisées rafraîchies par cron, sans suivi d'invalidation
- Un script de purge qui supprime ligne à ligne et verrouille la table

C'est cette base qui fournit les requêtes réécrites en M06, et qui est migrée en M12.



BLOC 1.2

Étendre PostgreSQL : quand oui, quand non

Une extension n'est pas une réponse universelle. Savoir dire non fait partie du métier.

Ce que l'on garde en restant sur PostgreSQL

- **Le SQL, les compétences, les outils**

Aucune requête à réapprendre, aucun pilote à changer, les procédures d'exploitation restent valides.

- **La jointure native au référentiel**

Une mesure et sa machine se joignent dans la même requête, dans la même transaction.

- **Les garanties transactionnelles**

L'ingestion et les corrections bénéficient des mêmes garanties que le reste du système.

- **Un seul moteur à exploiter**

Une seule chose à sauvegarder, sécuriser, superviser, auditer et faire monter de version.

Quand la réponse est non

Quatre situations où ajouter une base dédiée est le bon choix, et où l'extension ne l'est pas.

- **Cardinalité de séries extrême**

Plusieurs millions de séries actives simultanément : le coût de planification et de segmentation devient prohibitif.

- **Ingestion à plusieurs millions de points par seconde**

Sans aucun besoin relationnel ni transactionnel en face.

- **Charge purement analytique**

Aucune dimension transactionnelle : un entrepôt colonne pur sera plus efficace et moins cher.

- **Rétention de quelques heures**

Si la donnée ne survit pas à la journée, une file de messages suffit : il n'y a pas de base à concevoir.

Les alternatives

InfluxDB

Le bon choix quand

Cardinalité très élevée, écriture massive, aucun besoin de jointure relationnelle.

Ce que cela coûte

Langage propre, écosystème séparé, jointure au métier à faire ailleurs.

ClickHouse

Le bon choix quand

Analytique pur sur très gros volumes, agrégations sur des milliards de lignes.

Ce que cela coûte

Modèle transactionnel faible, mises à jour coûteuses, seconde exploitation à tenir.

pg_partman

Le bon choix quand

Partitionnement simple, volumes modérés, aucun besoin de compression ni d'agrégat incrémental.

Ce que cela coûte

Tout ce qui dépasse le partitionnement reste à écrire et à maintenir soi-même.

Sur des mesures et des cas réels, pas sur du dogme. Un outil se situe, il ne se défend pas.



BLOC 1.3

Les douze concepts

Posés une fois, réutilisés partout. Cochés à chaque point d'acquis.

Douze notions, quatorze modules

01

Série et point

02

Mesure ou événement

03

Cardinalité

04

Hypertable

05

Chunk

06

Dimension

07

Exclusion de chunks

08

Hypercore

09

Seau temporel

10

Agrégat continu

11

Politique et job

12

Agrégabilité

La donnée tardive complète la liste : elle est introduite en M05, une fois l'ingestion mesurée.

Ce que l'on stocke

01

Série et point

Une série est identifiée par une combinaison de clés. Un point est une valeur datée dans cette série.

02

Mesure ou événement

Une mesure est un état échantillonné, un événement est une transition datée. Ils ne se modélisent pas pareil.

03

Cardinalité de séries

Nombre de combinaisons distinctes de clés. Détermine la faisabilité plus sûrement que le volume brut.

04

Hypertable

Table logique unique, partitionnée automatiquement, vue comme une table ordinaire par le SQL applicatif.

Comment c'est découpé

05

Chunk

Partition physique. Unité de compression, de rétention, de verrouillage et d'exclusion.

06

Dimension

Axe de partitionnement. La dimension temporelle est obligatoire, la dimension spatiale est optionnelle.

07

Exclusion de chunks

Élimination des partitions hors périmètre, à la planification ou à l'exécution. Le mécanisme central du gain.

08

Hypercore

Rowstore et columnstore comme deux états successifs du même chunk, et non comme deux tables distinctes.

Comment c'est agrégé et entretenu

09

Seau temporel

Discrétisation régulière de l'axe du temps. Base de toute agrégation temporelle.

10

Agrégat continu

Vue matérialisée entretenue de façon incrémentale, capable de combiner matérialisé et données fraîches.

11

Politique et job

Mécanisme unique d'automatisation : compression, rétention, rafraîchissement et tâches maison.

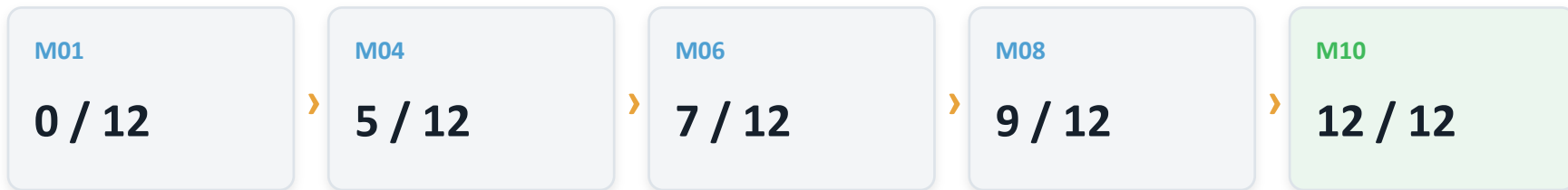
12

Agrégabilité

Une moyenne de moyennes est fausse, un percentile n'est pas agrégeable. D'où les états intermédiaires.

La carte des concepts

Un A3 distribué maintenant, coché à chaque point d'acquis. C'est l'instrument de continuité entre quinze modules.



Règle d'usage

- Une notion n'est cochée que lorsqu'elle a été instanciée sur MISTRAL, pas lorsqu'elle a été énoncée
- Le point d'acquis relit la carte et le fichier mesures.md, pas les slides
- Une notion non cochée au jalon prévu signale un retard réel, pas une impression



BLOC 1.4

Nomenclature, éditions, versions

Trois sujets administratifs qui décident de ce qui sera possible en production.

Trois noms, trois choses différentes

TimescaleDB

L'extension PostgreSQL. C'est elle que l'on installe, que l'on fait monter de version, et dont on lit les notes de publication.

TigerData

L'éditeur. Anciennement Timescale Inc., renommé en 2025. Le nom apparaît dans la documentation et les dépôts.

Tiger Cloud

Le service managé. Certaines fonctions n'existent que là — le stockage étagé, notamment, abordé en M10.

Conséquence pratique

Une recherche documentaire sur « Timescale » remonte trois générations de nommage. Vérifier la version à laquelle une page se rapporte avant de l'appliquer.

Apache 2.0 ou TSL : ce que le choix conditionne

Édition Apache 2.0

- Hypertables et partitionnement automatique
- time_bucket et fonctions de temps
- Exclusion de chunks
- Index et catalogue

Édition TSL

- Hypercore et columnstore
- Agrégats continus
- Politiques et ordonnanceur de jobs
- Toolkit et hyperfonctions

Ce qu'il faut retenir

La TSL est gratuite et son code est ouvert : elle n'interdit pas l'usage en production, elle interdit de revendre le produit comme service. Presque tout ce qui justifie cette formation est en TSL.

Versions et support

La matrice de support commande l'ordre des opérations d'une montée de version. Le sujet est repris en M13.

TimescaleDB 2.29

PostgreSQL 16, 17, 18

Version cible de la formation

TimescaleDB 2.28

PostgreSQL 15, 16, 17, 18

Dernière version couvrant PostgreSQL 15

Conséquence pour une instance en PostgreSQL 15

La montée vers 2.29 n'est pas directe : il faut d'abord monter PostgreSQL, puis l'extension. L'ordre inverse échoue, et il n'existe pas de retour arrière sur une montée d'extension.

Où en est-on

Acquis du module

- Nommer un problème observé
- Le rattacher à un concept
- Savoir dire quand c'est non

Carte des concepts

0 / 12

Douze notions nommées.
Aucune encore instanciée sur
MISTRAL.

Question de contrôle

Chaque participant énonce, pour son propre contexte, laquelle des six pathologies il observe aujourd'hui, et laquelle il n'observe pas. Une réponse « aucune » est une réponse valide, et elle mérite d'être discutée.

M02 · Déploiement et mise en service

1

Mettre en service

Instance épinglée à 2 vCPU et 8 Go, extension activée, workers vérifiés.

2

Poser la méthode de mesure

mesure.sh, médiane sur cinq exécutions, invalidation au-delà de 20 % d'écart-type.

3

Établir la ligne de base

Trois mesures de référence consignées dans mesures.md. Tout le reste s'y compare.

À vérifier avant de commencer

Les trois dumps du parcours amont sont restaurés, et le poste dispose de 60 Go libres. Un participant qui n'a pas restauré les jeux ne peut pas suivre M02.



MODULE M02

Déploiement, mise en service, méthode de mesure

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M01

- Six pathologies nommées, chacune rattachée à un module
- Les cas où la réponse est non, énoncés
- Douze concepts posés, carte A3 distribuée

Ce qui manque encore

- Aucune instance vérifiée : le poste a restauré des dumps, rien de plus
- Aucune mesure de référence : rien à quoi comparer
- Aucune notion instanciée sur MISTRAL : 0 / 12

Objectif du module passer d'un poste qui a des fichiers à un poste qui a une instance conforme et une ligne de base mesurée.

Agenda et objectifs

1 Déploiement et configuration

2 Méthode de mesure

L Atelier L01 · mise en service et ligne de base

✓ Point d'acquis

À l'issue du module

- Disposer d'une instance conforme, épinglée et vérifiée
- Savoir produire une mesure défendable, et savoir quand la jeter



BLOC 2.1

Déploiement et configuration

L'installation détaillée est en travail préparatoire. Ici, seulement ce qui décide de la suite.

Quatre modes, un seul choix à faire

Paquets Linux

Contrôle total sur la configuration et les montées de version. C'est le mode qui impose le plus de travail d'exploitation, et le seul qui le rende visible.

Image Docker

Le mode de la formation. Reproductible, jetable, épinglable en ressources. Attention aux entrées-sorties sur macOS et Windows, où tout transite par une machine virtuelle.

Chart Helm

Déploiement en cluster. La question n'est plus l'installation mais la persistance des volumes et le comportement au redémarrage d'un nœud.

Tiger Cloud

Service managé. Certaines fonctions n'existent que là — le stockage étagé, abordé en M10. En contrepartie, la configuration fine n'est plus accessible.

Activer l'extension, puis le vérifier

```
-- postgresql.conf : sans la premiere bibliotheque, CREATE EXTENSION echoue  
shared_preload_libraries = 'timescaledb,pg_stat_statements'
```

```
CREATE EXTENSION IF NOT EXISTS timescaledb;  
CREATE EXTENSION IF NOT EXISTS timescaledb_toolkit;  
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
```

```
-- les versions chargees, pas celles installees sur le disque  
SELECT extname, extversion FROM pg_extension ORDER BY extname;
```

Trois extensions, pas une

timescaledb pour le produit, le Toolkit pour les hyperfonctions de M07, et le suivi des requêtes pour la non-régression de M15. Les trois s'installent séparément, et les deux dernières s'oublient.

extversion, pas \dx

La commande \dx affiche ce qui est disponible. Seule extversion dit ce qui est réellement chargé. La bibliothèque se charge au démarrage : la modifier impose un redémarrage.

Les paramètres qui comptent vraiment

shared_buffers

Cache de pages. Décide si le chunk actif reste en mémoire — la règle centrale de M04.

work_mem

Mémoire par opération de tri ou de hachage. Une agrégation qui déborde sur disque coûte un ordre de grandeur.

maintenance_work_mem

Création d'index et compression. Sous-dimensionné, il allonge les bascules vers le columnstore.

timescaledb.max_background_workers

Nombre de workers pour les politiques. Sous-dimensionné, les jobs de M11 s'attendent en file.

max_worker_processes

Plafond global. Doit couvrir les workers TimescaleDB et le parallélisme PostgreSQL, sinon rien ne démarre.

Le paramétrage fin de l'ingestion (WAL, checkpoints, autovacuum par table) relève de M05 et de M15, une fois qu'il y a quelque chose à mesurer.

timescaledb-tune : ce qu'il règle, ce qu'il ne règle pas

Ce qu'il règle

- Les paramètres mémoire, à partir de la RAM détectée
- Le nombre de workers d'arrière-plan et de workers parallèles
- Quelques paramètres de planification

Ce qu'il ne règle pas

- Le WAL et les checkpoints, qui dominent l'ingestion
- L'autovacuum, en particulier ses réglages par table
- Le dimensionnement des chunks, des index et du pool de connexions

Ce qu'il faut retenir

C'est un point de départ correct, pas une configuration de production. Il fixe les paramètres qu'on aurait de toute façon fixés ; il ne touche pas ceux qui font mal.

Vérifier les workers d'arrière-plan

Toute l'automatisation de la formation en dépend : compression, rétention, rafraîchissement des agrégats, tâches maison. Un worker absent est un silence, pas une erreur.

```
SHOW timescaledb.max_background_workers;  
  
SELECT application_name, backend_type, state  
FROM   pg_stat_activity  
WHERE  application_name LIKE 'TimescaleDB%';
```

Le mode de défaillance à connaître dès maintenant

Si max_worker_processes ne couvre pas la somme des workers TimescaleDB et du parallélisme PostgreSQL, les politiques ne s'exécutent pas — et rien ne le signale. C'est la première panne du catalogue de M15, et elle commence ici.



BLOC 2.2

Méthode de mesure

Cinq jours de pédagogie reposent sur des chiffres. Sans discipline, ce ne sont que des chiffres.

Pourquoi une méthode, pas un chronomètre



Deux voisins, deux résultats

La même requête sur deux postes identiques donne 1,2 s et 4,8 s. Sans méthode, la salle passe vingt minutes à chercher pourquoi.



Une mesure prise trop tôt

Un autovacuum ou une création d'index encore en cours, et la mesure ne dit rien sur ce qu'on croit mesurer.



Une valeur unique

Une exécution ne dit rien de la dispersion. Le cache change tout entre la première et la deuxième.

La conséquence pédagogique

Toutes les décisions de la formation (intervalle de chunk, stratégie d'ingestion, segmentation du columnstore) sont justifiées par une mesure. Une mesure non défendable rend la décision non défendable, et l'atelier devient une récitation.

Les cinq règles

1

Cinq exécutions, médiane retenue

La moyenne se laisse déplacer par une exécution aberrante. La médiane non.

2

Écart-type au-delà de 20 % : mesure jetée

Ce n'est pas une mesure bruitée, c'est l'absence de mesure. Relancer à froid.

3

On compare des rapports, pas des durées

Le rapport entre deux configurations se transpose. La durée absolue, non.

4

EXPLAIN avec ANALYZE et BUFFERS, TIMING OFF

TIMING ON seulement pour isoler un nœud coupable : il fausse le total.

5

Tout est consigné dans mesures.md

C'est ce fichier qu'on relit au point d'acquis, pas les slides.

Le harnais en pratique

```
$ ./mesure.sh requetes/R2-fenetre-3j.sql
```

```
run 1 : 1372 ms      run 4 : 1361 ms
```

```
run 2 : 1358 ms      run 5 : 1379 ms
```

```
run 3 : 1365 ms
```

```
mediane   : 1365 ms
```

```
ecart-type: 8 ms    (0,6 %)  -> mesure retenue
```

Ce que le harnais garantit

Un protocole identique pour tous : même nombre d'exécutions, même mise en cache, même façon de rapporter.

Ce qu'il ne garantit pas

Que le poste voisin ait la même vitesse de disque. D'où la règle 3 : on compare des rapports.

Ce qu'un environnement de salle ne permet pas

Docker Desktop sur macOS et Windows

Les entrées-sorties transitent par une machine virtuelle et sont plusieurs fois plus lentes qu'en natif. Les ateliers de débit (M05) et de compression (M09) en sont directement affectés : les durées absolues qui y seront mesurées ne sont pas transposables en production.

Ce qui reste valide

Les rapports entre configurations, les plans d'exécution, les nombres de chunks, les ratios de compression, les volumes.

Ce qui ne l'est pas

Toute durée absolue, tout débit en points par seconde, toute extrapolation directe vers la production.

mesures.md, le journal de bord

```
## M02 – ligne de base (2 vCPU / 8 Go, Docker, ext 2.29, table ordinaire 5 j)
```

requete	mediane	ecart-type	note	
-----	-----	-----	-----	
R1 dernier point	5349 ms	2,3 %	table brute	
R2 fenetre 3 jours	1365 ms	0,6 %	table brute	
R3 agregation j1-j5	1372 ms	1,1 %	table brute	

Pourquoi un fichier plutôt que des notes

- Chaque point d'acquis relit ce fichier : c'est la preuve de ce qui a été établi, pas de ce qui a été dit
- Il est versionné avec le reste : la mesure et la décision qu'elle justifie vivent au même endroit
- En fin de formation, il constitue le premier élément du plan d'application du participant



ATELIER L01

Mise en service et ligne de base

Le jeu MISTRAL est restauré. Il s'agit d'en faire un environnement de travail vérifié.

Brief

SOCLE · pour tous

- Démarrer l'instance épinglée à 2 vCPU et 8 Go, swap désactivé
- Activer les trois extensions et contrôler les versions chargées
- Appliquer la configuration recommandée, vérifier l'activité des workers
- Exécuter `measure.sh` sur les trois requêtes de référence
- Relever la taille de la table et de ses index

EXTENSION · pour aller plus loin

- Comparer la sortie de `timescaledb-tune` à la configuration appliquée, et expliquer chaque écart
- Provisionner un service Tiger Cloud, comparer sa configuration par défaut
- Mesurer l'effet de `shared_buffers` sur la requête la plus lente, en trois valeurs

Réussi si les trois extensions sont chargées, les workers sont actifs, et les trois médianes sont consignées avec un écart-type sous 20 %.

Pièges et indices

La limite du conteneur est ignorée

- Sur macOS et Windows, `--memory` ne fait effet que si la machine virtuelle Docker Desktop est plus grande. Vérifier la VM avant le conteneur.

\dx ne dit pas ce qui est chargé

- Il affiche ce qui est disponible. Lire `extversion` dans `pg_extension`, dans la base où l'on travaille.

L'écart-type dépasse 20 %

- Une indexation ou un autovacuum tourne encore. Attendre que l'activité retombe : ne pas moyenniser du bruit.

Les workers ne démarrent pas

- Vérifier `max_worker_processes` avant `timescaledb.max_background_workers`. Le plafond global l'emporte, en silence.

État attendu

Livrable du module

- Instance conforme et vérifiée
- mesures.md avec trois références
- État de reprise mistral-M02

Carte des concepts

0 / 12

Aucune notion instanciée. M02
outil, il n'enseigne pas encore le
produit.

Resynchronisation

- Un participant dont l'instance n'est pas conforme restaure l'état mistral-M02 et repart de là
- Un participant dont les trois mesures manquent ne pourra rien comparer en M04 : les produire avant la pause
- Un écart-type durablement élevé sur un poste signale un environnement inadapté, pas une erreur de manipulation

M03 · Modélisation des données temporelles

L'instance tourne et la ligne de base est posée. Sur quel schéma, on l'ignore encore.

1

Étroit, large, intermédiaire

Trois modèles, des effets opposés sur le volume, la compression et l'évolutivité du schéma.

2

La clé d'identification de série

Le choix le plus lourd du module : il détermine le segmentby du columnstore, sept modules plus loin.

3

Le schéma cible MISTRAL

Produit en atelier, justifié par écrit, et utilisé par tous les modules suivants.

Une décision qui engage

Le schéma produit en M03 n'est pas juste un exercice : c'est celui que tous les ateliers suivants utilisent, jusqu'à la migration de M12.



MODULE M03

Modélisation des données temporelles

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M02

- Instance conforme, épinglée, workers vérifiés
- Ligne de base : trois médianes consignées dans mesures.md
- Volume de départ relevé, dénominateur du ratio de M09

Ce qui manque encore

- Les trois requêtes portent sur un schéma que personne n'a discuté
- Aucune hypertable : mesures est encore une table ordinaire
- Le référentiel et les mesures ne sont pas reliés

Objectif du module produire le schéma que tous les ateliers suivants utiliseront, jusqu'à la migration de M12.

Agenda et objectifs

- 1 Anatomie d'une hypertable
- 2 Modèle étroit, large, intermédiaire
- 3 Types, attributs variables, référentiel
- L Atelier L02 · schéma cible MISTRAL
- ✓ Point d'acquis

À retenir dès maintenant la clé d'identification de série choisie ici détermine le segmentby du columnstore, six modules plus loin.



BLOC 3.1

Anatomie d'une hypertable

Ce que l'extension ajoute réellement au partitionnement déclaratif de PostgreSQL.

BLOC 3.1

Deux façons de créer, une seule à privilégier

Création directe → la voie recommandée depuis les versions récentes

```
CREATE TABLE mesures (  
  ts          TIMESTAMPTZ          NOT NULL,  
  series_id  INTEGER              NOT NULL,  
  valeur     DOUBLE PRECISION NOT NULL  
) WITH (timescaledb.hypertable,  
        timescaledb.partition_column = 'ts',  
        timescaledb.chunk_interval  = '7 days');
```

Conversion d'une table existante → pour migrer, pas pour concevoir

```
SELECT create_hypertable('mesures', by_range('ts', INTERVAL '7 days'),  
                        migrate_data => true);  
  
-- lent et verrouillant sur un gros volume : sujet du module M12
```

Hypertable ou partitionnement déclaratif

Partitionnement déclaratif seul

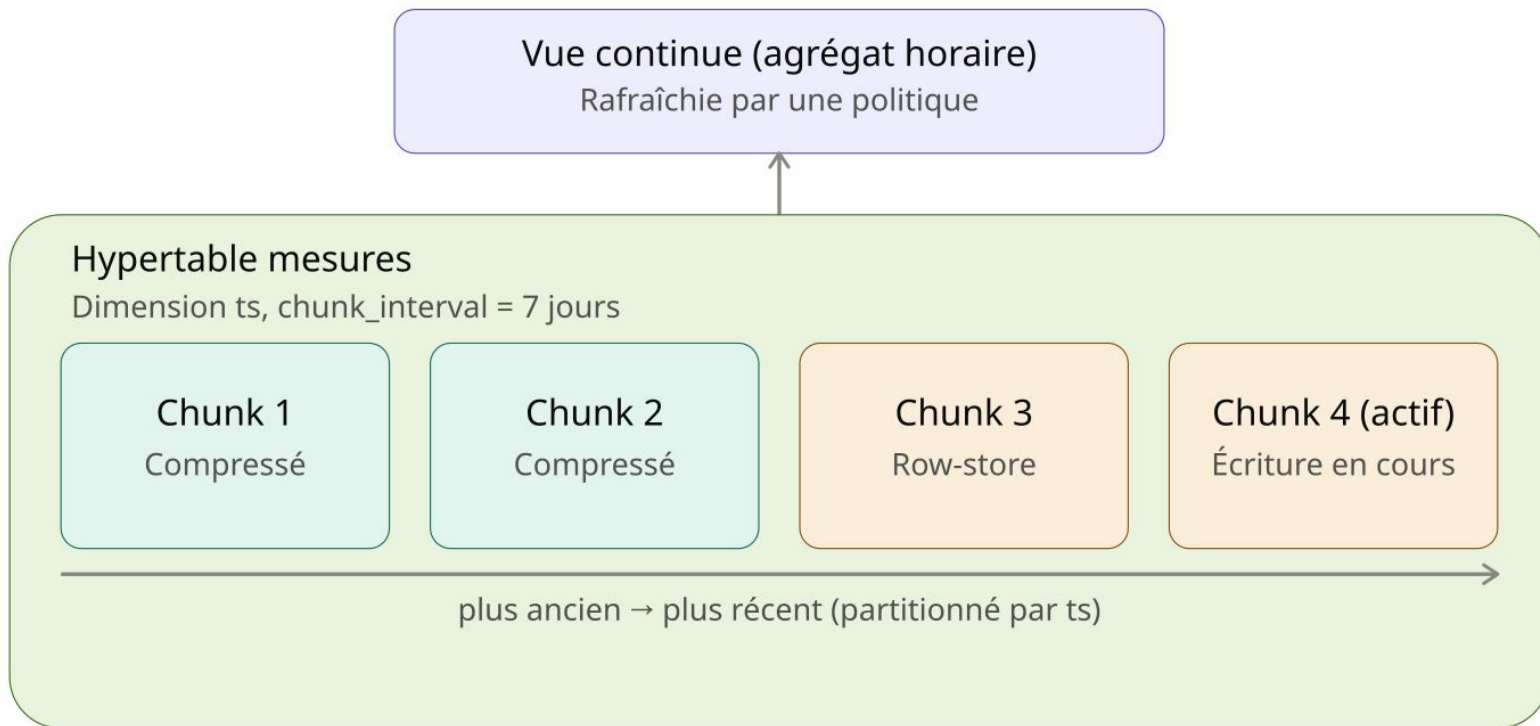
- Les partitions sont créées à la main, ou par un script maison
- Chaque nouvelle période exige une intervention ou un cron
- Aucune compression, aucune rétention, aucun agrégat incrémental

Hypertable

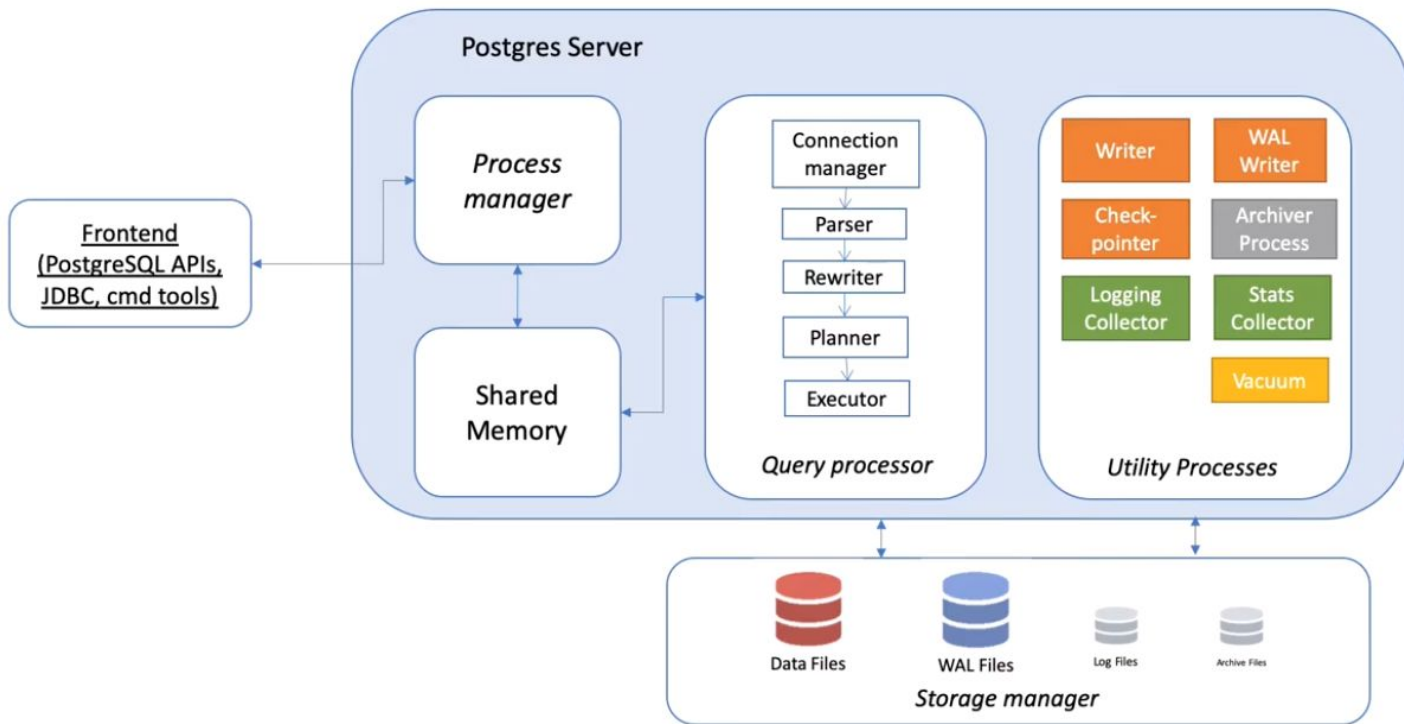
- Les chunks sont créés à l'insertion, sans intervention
- Le chunk devient l'unité de compression, de rétention et de verrouillage
- Les politiques, les agrégats continus et le columnstore s'y accrochent

La base MISTRAL actuelle utilise le partitionnement déclaratif. Une table déjà partitionnée ne peut pas être convertie directement → contrainte reprise en M12.

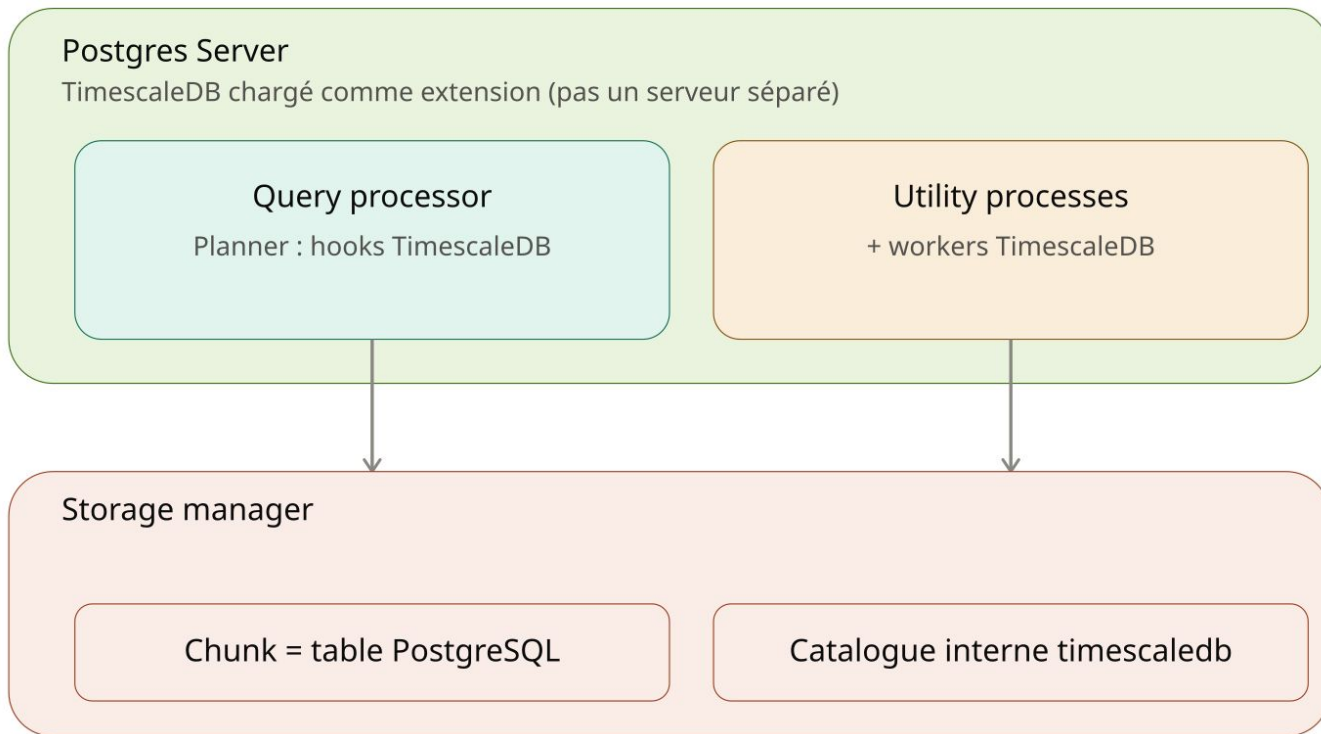
Hypertable : Illustration



Hypertable et architecture Postgres



Hypertable et architecture Postgres



BLOC 3.1

Inspecter le catalogue

Ces trois requêtes reviennent tous les jours de la formation. Elles constituent le premier élément du jeu de diagnostic construit en M15.

```
-- quelles hypertables, combien de dimensions
SELECT hypertable_name, num_dimensions
FROM   timescaledb_information.hypertables;

-- les cinq chunks les plus récents et leurs bornes
SELECT chunk_name, range_start, range_end
FROM   timescaledb_information.chunks
WHERE  hypertable_name = 'mesures'
ORDER  BY range_start DESC LIMIT 5;

-- le volume, décompose
SELECT * FROM hypertable_detailed_size('mesures');
```

Contraintes et clés : la règle et ses raisons

Toute contrainte d'unicité doit contenir la colonne de partitionnement

- L'unicité globale ne peut pas être garantie sans lire tous les chunks. Ce n'est pas une limitation arbitraire, c'est le prix du partitionnement.

La colonne de temps doit être NOT NULL

- Une ligne sans date n'a pas de chunk d'accueil.

Une clé étrangère entre deux hypertables n'est pas autorisée

- Toutes les autres combinaisons le sont, y compris une hypertable qui référence le référentiel métier.

Une table déjà partitionnée ne peut pas être convertie

- Partitionnement déclaratif ou héritage : il faut recréer, pas convertir.



BLOC 3.2

Modèle étroit, large, intermédiaire

Trois formes possibles pour la même donnée, et des conséquences opposées.

Le modèle étroit

```
CREATE TABLE mesures (  
  ts          TIMESTAMPTZ      NOT NULL,  
  series_id  INTEGER          NOT NULL,  
  -- une ligne = un signal, un instant  
  valeur     DOUBLE PRECISION NOT NULL  
) WITH (timescaledb.hypertable);
```

Le bon choix quand

Le parc de signaux évolue, les signaux sont hétérogènes, et les requêtes portent sur un signal à la fois plutôt que sur une machine entière.

Conséquences

- Ajouter un capteur ne touche pas au schéma
- Compression très efficace : peu de colonnes, forte redondance
- Croiser deux signaux au même instant impose une jointure ou un pivot

Le modèle large

```
CREATE TABLE mesures (  
  ts          TIMESTAMPTZ NOT NULL,  
  machine_id  INTEGER      NOT NULL,  
  puissance_kw DOUBLE PRECISION,  
  vitesse_vent DOUBLE PRECISION,  
  temp_nacelle DOUBLE PRECISION,  
  -- ... autres colonnes  
) WITH (timescaledb.hypertable);
```

Le bon choix quand

Le parc est figé, les signaux arrivent au même instant, et les requêtes portent sur la machine entière.

Conséquences

- Croiser plusieurs signaux au même instant est immédiat
- Volume de lignes divisé par le nombre de signaux
- Ajouter un type de capteur impose ALTER TABLE et coordination applicative
- Colonnes creuses : les NULL se paient en stockage et en compression

Le modèle intermédiaire, par famille de signaux

```
-- une hypertable par famille, pas par signal
CREATE TABLE mesures_production (
  ts          TIMESTAMPTZ NOT NULL,
  machine_id  INTEGER      NOT NULL,
  puissance_kw DOUBLE PRECISION,
  energie_kwh DOUBLE PRECISION
) WITH (timescaledb.hypertable);

-- puis mesures_meteo, mesures_mecanique, ...
```

Souvent le bon compromis industriel — mais il faut savoir dire pourquoi, pas le choisir par défaut.

Conséquences

- Les signaux qui arrivent ensemble et se lisent ensemble vivent ensemble
- Chaque famille a sa fréquence, sa rétention et sa segmentation propres
- Un nouveau signal reste local à sa famille
- Une nouvelle famille est un vrai travail de conception, pas un ALTER

BLOC 3.2

Les trois modèles, sur les mêmes critères

Critère	Étroit	Large	Intermédiaire
Nombre de lignes	très élevé	faible	moyen
Ratio de compression	excellent	moyen	bon
Ajout d'un signal	aucun impact	ALTER TABLE	local à la famille
Croiser deux signaux	jointure ou pivot	immédiat	immédiat dans la famille
Lisibilité des requêtes	faible	forte	moyenne

La clé d'identification de série

C'est la décision la plus lourde du module. Elle engage six modules plus loin.

series_id INTEGER

Entier, avec table de correspondance. Compacte, comparaison rapide, compression maximale.

(site, machine, signal)

Clé composite lisible. Plus volumineuse, mais elle porte le sens et évite une jointure.

tag TEXT

Identifiant textuel répété à chaque ligne. Lisible, et coûteux 190 millions de fois.

Pourquoi ce choix ne se rattrape pas facilement

La clé de série devient le segmentby du columnstore en M09. Sa cardinalité y détermine directement le ratio de compression et la vitesse des requêtes filtrées. La changer après coup impose de décompresser et de réécrire tous les chunks.



BLOC 3.3

Types, attributs variables, référentiel

Trois questions qui remontent systématiquement en industrie, et leurs réponses.

Types : trois décisions à ne pas différer

timestampz, jamais timestamp

- Sans fuseau, la même valeur signifie deux instants différents selon le serveur. Les agrégations journalières de M06 en dépendent directement.

double precision ou numeric

- numeric est exact et lent, et compresse mal. Pour de la télémétrie, double precision est presque toujours le bon choix — pour de la facturation, presque jamais.

Les unités appartiennent au référentiel

- Stocker une valeur sans son unité impose de la connaître ailleurs. La stocker à chaque ligne la paie 190 millions de fois.

BLOC 3.3

Mesures et événements : deux hypertables

Une mesure est un état échantillonné, un événement est une transition datée. Même axe de temps, modélisations opposées.

```
CREATE TABLE evenements (  
  ts          TIMESTAMPTZ NOT NULL,  
  machine_id  INTEGER      NOT NULL,  
  type        TEXT         NOT NULL,    -- alarme, maintenance, arret  
  attributs   JSONB         -- charge utile variable  
) WITH (timescaledb.hypertable);
```

Recroiser les deux : la mesure au moment de l'événement

```
SELECT e.ts, e.type, m.valeur  
FROM   evenements e  
CROSS JOIN LATERAL (  
  SELECT valeur FROM mesures  
  WHERE  series_id = e.machine_id AND ts <= e.ts  
  ORDER BY ts DESC LIMIT 1) m;
```

JSONB : ce qu'il apporte, ce qu'il coûte

Ce qu'il apporte

- Des attributs variables sans migration de schéma
- Une charge utile hétérogène par type d'événement
- Une indexation possible sur les clés fréquentes

Ce qu'il coûte

- Les noms de clés sont stockés à chaque ligne
- La compression y est nettement moins bonne qu'en colonnes typées
- Le typage se perd : la validation remonte dans l'applicatif

La règle applicable

JSONB pour les événements, dont la charge utile est réellement variable et le volume faible. Colonnes typées pour les mesures, dont le volume est élevé et la structure stable. L'extension du lab mesure l'écart.

BLOC 3.3

Le référentiel change, les mesures restent

Un capteur est remplacé le 12 octobre. À quelle machine appartiennent les points du 10 octobre ? La question n'a de réponse que si le référentiel est daté.

```
CREATE TABLE affectation_capteur (  
  series_id INTEGER NOT NULL,  
  machine_id INTEGER NOT NULL,  
  debut TIMESTAMPTZ NOT NULL,  
  fin TIMESTAMPTZ, -- NULL = courante  
  EXCLUDE USING gist (series_id WITH =,  
                      tstzrange(debut, fin) WITH &&  
);
```

```
SELECT m.ts, a.machine_id, m.valeur  
FROM   mesures m  
JOIN   affectation_capteur a  
      ON a.series_id = m.series_id  
AND    m.ts >= a.debut  
AND    (a.fin IS NULL OR m.ts < a.fin);
```

Ne pas dénormaliser par réflexe

Recopier machine_id dans mesures rend la jointure inutile — et rend impossible la correction d'une affectation erronée sans réécrire des millions de lignes.



ATELIER L02

Schéma cible MISTRAL

Le seul atelier de la formation dont le livrable est une décision, et non une mesure.

Brief

SOCLE · pour tous

- Compléter le squelette fourni : trancher les six points de décision marqués, avec une ligne de justification chacun
- Créer les hypertables sur le schéma retenu, intervalle par défaut — M04 y reviendra
- Écrire la jointure mesure vers référentiel retournant site, machine et libellé de signal
- Estimer le volume à 3 ans sur l'échelle de production

EXTENSION · pour aller plus loin

- Confronter deux propositions de sous-groupes et arbitrer au tableau
- Modéliser un capteur remplacé en cours de période, et retrouver la bonne machine pour chaque point
- Comparer JSONB et trois colonnes typées sur 10 M lignes, avant et après compression
- Écrire la variante en modèle large et mesurer l'écart de volume

Réussi si le schéma absorbe un nouveau type de capteur sans ALTER TABLE, la jointure au référentiel retourne le résultat attendu, et l'estimation à 3 ans est posée avec ses hypothèses écrites.

Pièges et indices

La clé primaire est refusée

Elle doit contenir ts. Un PRIMARY KEY (series_id) seul échoue, et le message ne dit pas pourquoi de façon évidente.

Le squelette propose un choix par défaut

Il n'est pas le bon choix : il est le choix le plus courant. Chaque point de décision attend une justification, pas une validation.

L'estimation à 3 ans est faite sur l'échelle d'atelier

190 M points sur 45 jours n'est pas MISTRAL en production. L'estimation porte sur 4 000 signaux à 0,1 Hz.

La jointure au référentiel ignore la date

Sans les bornes debut et fin, un capteur remplacé fausse tout l'historique en silence.

État attendu

Livrable du module

- schema.sql, arbitrages justifiés en commentaires
- Deux hypertables créées et chargées : mesures et evenements
- État de reprise mistral-M03

Carte des concepts

2 / 12

Série et point · Hypertable
Deux notions instanciées sur
MISTRAL.

Resynchronisation

- Un participant dont le schéma diverge de la variante de référence restaure l'état mistral-M03
- La variante de référence n'est pas la bonne réponse : c'est celle que la chaîne d'instantanés implémente
- Un participant qui a retenu un autre modèle le documente dans son plan d'application, et poursuit sur la référence

M04 · Chunks, index, exclusion

Le schéma est posé, les hypertables tournent sur l'intervalle par défaut. Est-ce le bon ?

1

Dimensionner les chunks

La règle : le chunk actif et ses index tiennent en mémoire. L'erreur la plus coûteuse du produit.

2

Index et exclusion

Ce qu'il faut créer, ce qu'il ne faut surtout pas créer, et comment lire l'exclusion dans un plan.

3

Trancher par la mesure

Trois intervalles, trois profils de requête, un intervalle retenu et justifié.

La ligne de base va enfin servir

Les trois médianes de M02 deviennent le point de comparaison des neuf mesures de M04. C'est le premier gain chiffré de la formation.



MODULE M04

Chunks, index, exclusion

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M03

- schema.sql, six décisions tranchées et justifiées
- Les hypertables existent et sont chargées
- La jointure au référentiel est datée, donc correcte

Ce qui manque encore

- L'intervalle de chunk est celui du squelette : aucune mesure ne le justifie
- Aucun index n'a été ajouté au-delà de ceux créés par défaut
- Personne n'a encore lu un plan d'exécution sur hypertable

Objectif du module remplacer une valeur par défaut par une décision mesurée — et savoir la défendre devant un plan d'exécution.

Agenda et objectifs

1 Dimensionner les chunks

2 Partitionnement spatial

3 Index et exclusion de chunks

L Atelier L03 · dimensionnement comparé

✓ Point d'acquis

L'erreur la plus coûteuse du produit se commet en cinq secondes, à la création, et se répare en réécrivant tous les chunks.



BLOC 4.1

Dimensionner les chunks

Une règle, deux effets de bord opposés, et une mesure pour trancher.

Un chunk est une table

Rien de magique : chaque chunk est une table PostgreSQL ordinaire, avec ses index et ses statistiques propres.

```
SELECT show_chunks('mesures');

SELECT c.chunk_name, c.range_start, c.range_end, pg_size_pretty(d.total_bytes) AS taille
FROM timescaledb_information.chunks c
JOIN chunks_detailed_size('mesures') d ON d.chunk_name = c.chunk_name
WHERE c.hypertable_name = 'mesures'
ORDER BY c.range_start DESC;
```

Unité de compression [M09](#)

Unité de rétention [M10](#)

Unité de verrouillage [M05](#)

Unité d'exclusion [ici](#)

Le chunk actif doit tenir en mémoire

L'ensemble des chunks activement écrits, index compris, doit tenir dans environ un quart de la mémoire disponible.

Pourquoi cette règle

Tant que le chunk actif et ses index sont en cache, une insertion touche des pages chaudes. Dès qu'ils débordent, chaque insertion provoque une lecture disque pour mettre à jour l'index.

Pourquoi un quart

Le reste de la mémoire sert aux lectures, au tri, à la maintenance et au système de fichiers. C'est un ordre de grandeur, pas une constante.

Sur des mesures, pas sur du dogme le quart est un point de départ. L'atelier vérifie s'il tient sur MISTRAL, et le corrige si les mesures le contredisent.

Appliquer la règle à MISTRAL

	Échelle d'atelier	Échelle de production
Séries actives	490	4 000
Points par jour	4,2 M	34,6 M
Octets par ligne, index compris	≈ 82	≈ 82
Volume par jour	≈ 350 Mo	≈ 2,8 Go
Mémoire de l'instance	8 Go	64 Go
Budget du chunk actif (un quart)	2 Go	16 Go
Intervalle théorique	≈ 6 jours	≈ 6 jours

Le résultat converge parce que la règle est un rapport : le volume et la mémoire croissent ensemble. Ce qui change à l'échelle de production, c'est qu'on dimensionne aussi le serveur.

Intervalle trop court

Le nombre de chunks explose



45 jours en chunks d'une heure font 1 080 partitions pour une seule hypertable. Le catalogue interne grossit d'autant.

Le temps de planification s'effondre



Le planificateur doit considérer chaque chunk avant de l'exclure. Sur une requête de quelques millisecondes, la planification devient le coût dominant.

Les statistiques se dispersent



Chaque chunk a ses propres statistiques, calculées sur peu de lignes. Les estimations de cardinalité se dégradent, et les plans avec elles.

La compression perd en efficacité



Les lots compressés de M09 sont bornés par le chunk. Un chunk minuscule produit des lots minuscules.

Intervalle trop long

Le chunk actif ne tient plus en mémoire



L'effet est brutal et non linéaire : le débit d'insertion s'effondre au moment précis où l'index déborde du cache.

L'exclusion perd sa granularité



Une requête sur trois jours lit un chunk de trente jours. Le mécanisme central du gain ne joue plus qu'à moitié.

La rétention devient grossière



On ne peut supprimer que des chunks entiers. Un chunk mensuel impose de conserver jusqu'à un mois de données au-delà de la durée voulue.

Les verrous s'élargissent



Compression, réindexation et suppression verrouillent un chunk entier. Plus il est gros, plus l'opération est visible en production.

Modifier l'intervalle après coup

```
SELECT set_chunk_time_interval('mesures', INTERVAL '2 days');

-- verification : la nouvelle valeur ne concerne que les chunks à venir
SELECT hypertable_name, column_name, time_interval
FROM   timescaledb_information.dimensions
WHERE  hypertable_name = 'mesures';
```

Ce qui s'applique immédiatement

Les chunks créés après l'appel. La transition est silencieuse : l'hypertable contient alors des chunks de deux tailles, ce qui est parfaitement licite.

Ce qui impose une reconstruction

Les chunks existants gardent leur intervalle. Les réaligner suppose de recréer l'hypertable et d'y recopier les données → c'est-à-dire une migration, sujet de M12.



BLOC 4.2

Partitionnement spatial

Une seconde dimension. Souvent proposée, rarement utile, et pas sur MISTRAL.

Une seconde dimension, par hachage

```
CREATE TABLE mesures (...)  
  WITH (timescaledb.hypertable,  
        timescaledb.partition_column = 'ts');  
  
-- ajout d'une dimension spatiale a 4 partitions  
SELECT add_dimension('mesures', by_hash('series_id', 4));
```

Quand elle aide réellement

- Plusieurs disques ou tablespaces à solliciter en parallèle
- Une charge d'insertion concurrente très forte sur une même fenêtre de temps

Quand elle fragmente pour rien

- Sur une instance à disque unique : elle multiplie les chunks sans rien paralléliser
- Quand les requêtes filtrent sur le temps : l'exclusion temporelle suffisait déjà

Sur MISTRAL, la réponse est non

Il faut savoir dire pourquoi, sinon quelqu'un l'ajoutera « pour la performance ».

1

Un seul volume de stockage

Le hachage réparti sur des partitions logiques, pas sur des disques. Sans disques multiples, il n'y a rien à paralléliser.

2

Les requêtes filtrent sur le temps

L'exclusion temporelle élimine déjà l'essentiel. La dimension spatiale n'ajoute une exclusion que si la requête filtre aussi sur la clé de hachage.

3

Le nombre de chunks est multiplié

Quatre partitions spatiales font quatre fois plus de chunks, avec tous les effets de l'intervalle trop court vus au bloc précédent.



BLOC 4.3

Index et exclusion de chunks

Le mécanisme central du gain, et les trois façons de le perdre sans s'en apercevoir.

Trois catégories d'index

Créés par défaut

- Un index sur la colonne de temps, en ordre décroissant
- Si une dimension spatiale existe, un index sur (dimension, temps)

À ajouter selon l'usage

- (series_id, ts DESC) pour la dernière valeur par série
- Un index partiel sur les lignes de qualité dégradée, si on les filtre souvent

À ne surtout pas créer

- Un index sur series_id seul : redondant avec le composite, et payé à chaque insertion
- Tout index posé avant un chargement massif : le créer après divise la durée

Chaque index se paie deux fois en volume sur disque, et en travail à chaque insertion. L'atelier de M05 le mesure.

Repérer l'exclusion dans un plan

```
EXPLAIN (ANALYZE, BUFFERS, TIMING OFF)
SELECT avg(valeur) FROM mesures
WHERE ts >= '2026-09-20' AND ts < '2026-09-23';
```

```
Aggregate (actual rows=1 loops=1)
  -> Append (actual rows=12700800 loops=1)
        -> Seq Scan on _hyper_1_18_chunk
        -> Seq Scan on _hyper_1_19_chunk
Planning Time: 0.412 ms
```

Ce qu'il faut lire

Deux chunks sous le nœud Append, sur une hypertable qui en compte sept. Les cinq autres ont été éliminés à la planification : ils n'apparaissent pas du tout. Un plan qui liste tous les chunks est un plan sans exclusion.

Exclusion à la planification, ou à l'exécution

À la planification

Les bornes sont des constantes connues du planificateur.
Les chunks écartés n'apparaissent pas dans le plan, et le temps de planification reste faible.

Le cas favorable : c'est celui qu'on cherche à obtenir.

À l'exécution

Les bornes viennent d'un paramètre, d'une requête préparée ou d'une sous-requête. Tous les chunks figurent dans le plan ; l'élimination se lit sur une ligne dédiée au démarrage.

Le résultat est correct, mais la planification a coûté le prix de tous les chunks.

Conséquence pour une application une requête préparée avec des bornes paramétrées bascule en exclusion à l'exécution. Sur une hypertable à mille chunks, la différence se voit à l'œil nu.

BLOC 4.3

Le prédicat qui annule tout

Les deux requêtes retournent le même résultat. Une seule permet l'exclusion.

Aucune exclusion, tous les chunks sont lus

```
WHERE date_trunc('day', ts) = '2026-10-01'
```

Exclusion à la planification

```
WHERE ts >= '2026-10-01' AND ts < '2026-10-02'
```

Pourquoi

Appliquer une fonction à la colonne de partitionnement la rend opaque au planificateur : il ne peut plus comparer le prédicat aux bornes des chunks. C'est la même requête pour le métier, ce n'est pas la même pour le planificateur.

Les quatre façons de perdre l'exclusion

Écriture fautive	Cause	Correction
<code>date_trunc('day', ts) = ...</code>	Fonction appliquée à la colonne	Réécrire en bornes : <code>ts >= X AND ts < Y</code>
<code>ts::date = '2026-10-01'</code>	Cast, donc fonction implicite	Même réécriture en bornes
<code>EXTRACT(hour FROM ts) = 14</code>	Prédicat non temporellement contigu	Filtrer d'abord sur une plage, extraire ensuite
<code>ts IN (SELECT ...)</code>	Bornes inconnues à la planification	Matérialiser les bornes, ou accepter l'exclusion à l'exécution

Ces quatre écritures sont dans le jeu de cinq requêtes de l'atelier. Une seule n'exclut rien du tout.



ATELIER L03

Dimensionnement comparé

Trois intervalles, trois profils de requête, un intervalle retenu et justifié par une mesure.

Brief

SOCLE • pour tous

- Trois hypertables sur les mêmes données : intervalles 1 jour, 7 jours, 30 jours
- Relever pour chacune le nombre de chunks, la taille du catalogue et le temps de planification
- Mesurer trois profils de requête : point récent, fenêtre de 3 jours, agrégation complète
- Trouver, parmi cinq requêtes, celle qui n'exclut aucun chunk — diagnostiquer et corriger
- Trancher l'intervalle MISTRAL et l'inscrire dans schema.sql avec la mesure

EXTENSION • pour aller plus loin

- Une quatrième hypertable à intervalle 1 heure : chiffrer l'effondrement de la planification par chunk
- Reproduire la perte d'exclusion avec une requête préparée, et lire l'élimination au démarrage
- Mesurer l'effet d'un index inutile sur le débit d'insertion, en trois configurations
- Recalculer l'intervalle théorique à l'échelle de production et le confronter à celui de l'atelier

Réussi si l'intervalle retenu est justifié par une mesure consignée, et le participant sait montrer l'exclusion de chunks dans un plan à un voisin.

Le tableau à remplir

Neuf mesures, plus trois relevés structurels. Le tableau est fourni pré-rempli d'en-têtes dans mesures.md.

Grandeur relevée	1 jour	7 jours	30 jours
Nombre de chunks			
Temps de planification			
R1 · point le plus récent			
R2 · fenêtre de 3 jours			
R3 · agrégation complète			

La colonne qui gagne sur R2 n'est pas celle qui gagne sur R3. C'est l'objet de la décision : arbitrer, pas optimiser un seul chiffre.

Pièges et indices

Le temps de planification n'apparaît pas

- C'est lui qui trahit la prolifération de chunks, pas le temps d'exécution. Le relever explicitement, il se lit sur la ligne Planning Time.

Les trois hypertables sont créées mais restent vides

- Copier les données prend du temps. Utiliser la fenêtre réduite prévue dans l'énoncé plutôt que les 45 jours complets.

Changer l'intervalle ne change rien

- `set_chunk_time_interval` ne touche pas les chunks existants. Pour comparer, il faut trois hypertables distinctes, pas trois réglages successifs.

La requête fautive semble rapide

- Sur 45 jours et sept chunks, lire tout n'est pas dramatique. C'est le plan qu'il faut regarder, pas le chronomètre — et l'extrapoler à mille chunks.

État attendu

Livrable du module

- Intervalle tranché, inscrit et mesuré
- mesures.md enrichi de neuf mesures
- État de reprise mistral-M04

Carte des concepts

5 / 12

Ajout de chunk, dimension et exclusion de chunks.

Resynchronisation

- Un participant qui n'a pas terminé les neuf mesures restaure l'état mistral-M04 et reprend les mesures manquantes en pause
- Le premier gain chiffré est ici : R2 sur l'hypertable contre la même requête sur une copie ordinaire des 45 jours. La ligne de base M02 (5 jours) ne montre aucun gain, et c'est attendu
- Un intervalle retenu sans mesure est un intervalle non tranché : le signaler, ne pas le laisser passer

CE QUI SUIV

M05 · Ingestion à haut débit

Le schéma est dimensionné. On peut désormais y écrire vite, ou lentement.

1

Un ordre de grandeur

Entre l'INSERT unitaire et le COPY binaire, le rapport dépasse dix. Il se mesure, il ne se récite pas.

2

Nommer la ressource limitante

Trouver le point de rupture et dire ce qui sature, sur la base d'une mesure et non d'une intuition.

3

Mesurer le retard d'arrivée

Une valeur produite ici, et dont M08 a besoin pour dimensionner ses fenêtres de rafraîchissement.

Un index de trop se paie à l'écriture

L'extension de L03 mesure le coût d'un index inutile sur le débit d'insertion. C'est la même mesure que M05 reprend à l'échelle, sur trois stratégies d'écriture.



MODULE M05

Ingestion à haut débit

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M04

- Intervalle de chunk tranché, et prouvé par neuf mesures
- L'exclusion se lit dans un plan, et ses trois régimes sont connus
- Premier gain chiffré face à la ligne de base de M02

Ce qui manque encore

- Tout a été mesuré en lecture ; rien en écriture
- Personne ne sait combien de points par seconde cette instance accepte
- Le retard d'arrivée n'est pas mesuré — M08 en aura besoin

Objectif du module obtenir un ordre de grandeur entre la stratégie naïve et la bonne, et savoir nommer ce qui limite.

Agenda et objectifs

- 1 Mécanique de l'écriture
- 2 Idempotence, backfill, données tardives
- 3 Sources externes · démonstration guidée
- L Atelier L04 · montée en charge
- ✓ Point d'acquis

Une valeur produite ici sert ailleurs le retard d'arrivée mesuré dans l'atelier dimensionne les fenêtres de rafraîchissement de M08.



BLOC 5.1

Mécanique de l'écriture

Où passe réellement le temps d'une insertion, et ce que cela commande.

Où passe le temps d'un INSERT unitaire

1

Aller-retour réseau

Souvent le poste dominant. En local il se compte en dizaines de microsecondes, à travers un réseau d'entreprise en millisecondes.

2

Analyse et planification

Chaque instruction est analysée, planifiée, et le chunk d'accueil est résolu.

3

Écriture et journal

La ligne est écrite dans la page, l'enregistrement de journal est produit et validé.

4

Mise à jour des index

Chaque index est parcouru et modifié. Un index de trop se paie ici, à chaque ligne.

Les points 1 et 2 se paient une fois par instruction. Les points 3 et 4 se paient une fois par ligne. Tout le reste du bloc découle de cette dissymétrie.

Insertion par lots : amortir l'instruction

```
-- un aller-retour, une analyse, une planification pour 1 000 lignes
INSERT INTO mesures (ts, series_id, valeur) VALUES
  ('2026-10-01 08:00:00+02', 137, 1842.5),
  ('2026-10-01 08:00:10+02', 137, 1839.2),
  ... -- 998 autres lignes
  ('2026-10-01 10:46:30+02', 137, 1791.4);
```

Ce que le lot amortit

L'aller-retour réseau, l'analyse et la planification — tout ce qui se paie par instruction.

Ce qu'il n'amortit pas

L'écriture, le journal et la mise à jour des index. Ce coût reste proportionnel au nombre de lignes.

Le plateau au-delà d'une certaine taille de lot, le gain cesse : le coût par ligne domine. L'extension de l'atelier trace la courbe et situe le plateau.

COPY : le levier principal

```
-- flux continu, pas d'analyse par ligne, chemin d'écriture optimise
COPY mesures (ts, series_id, valeur)
FROM STDIN WITH (FORMAT csv);

-- format binaire : pas de conversion texte, gain supplémentaire
COPY mesures FROM STDIN WITH (FORMAT binary);
```

Aucune analyse par ligne

Le flux est décodé, pas planifié ligne à ligne.

Chemin d'écriture dédié

L'insertion en masse emprunte un code plus court que l'INSERT.

Format binaire

Supprime la conversion texte vers type. Impose un client capable de le produire.

À retenir créer les index après le chargement plutôt qu'avant divise la durée d'un chargement initial. C'est vrai ici comme en M12.

Les trois stratégies, en ordres de grandeur

Stratégie	Débit attendu	Rapport	Ce qui limite
INSERT unitaire	quelques milliers/s	× 1	Aller-retour réseau
INSERT par lots de 1 000	quelques dizaines de milliers/s	× 20 à 30	Journal et index
COPY binaire	quelques centaines de milliers/s	× 100 et plus	Disque et journal

Ces chiffres sont des ordres de grandeur, pas des cibles

Le rapport entre la première et la dernière ligne dépend massivement de la latence réseau : de l'ordre de 20 en local, souvent au-delà de 100 à travers un réseau d'entreprise. L'atelier mesure le rapport réel de la salle.

Transactions, journal, et le compromis de durabilité

```
-- mesurer le volume de journal produit par une operation
SELECT pg_current_wal_lsn() AS avant \gset
-- ... executer le chargement ...
SELECT pg_size_pretty(
    pg_wal_lsn_diff(pg_current_wal_lsn(), :'avant')) AS wal_produit;
```

Taille de transaction

Une transaction par ligne provoque une validation par ligne. Une transaction trop large retarde la visibilité et allonge la reprise après incident.

Hypertable non journalisée

Supprime la production de journal, donc le premier goulet d'un chargement initial. Les données sont perdues en cas d'arrêt brutal : réservé aux chargements rejouables.

Le volume de journal se relève par différence de position, jamais par la taille du répertoire : celui-ci est recyclé en continu.



BLOC 5.2

Idempotence, backfill, données tardives

Trois situations où l'ingestion cesse d'être un simple problème de débit.

Rendre une ingestion idempotente

Un producteur qui rejoue un lot après incident est la règle, pas l'exception. Sans contrainte, le rejeu produit des doublons silencieux.

```
-- la contrainte doit contenir la colonne de partitionnement
ALTER TABLE mesures
  ADD CONSTRAINT mesures_uk UNIQUE (series_id, ts);

INSERT INTO mesures (ts, series_id, valeur)
VALUES (...)
ON CONFLICT (series_id, ts) DO NOTHING;
```

Le prix à connaître

L'index unique est maintenu à chaque insertion, et il s'ajoute aux index de lecture. Le surcoût se mesure — c'est l'objet d'une extension de l'atelier. DO NOTHING ignore le doublon ; DO UPDATE corrige la valeur, et coûte davantage.

Backfill sans dégrader l'ingestion courante

Charger de l'historique pendant que le flux temps réel continue : deux écritures qui se disputent la même instance.

● Le backfill touche des chunks froids

Ces chunks ne sont plus en cache. Chaque lot provoque des lectures disque qui entrent en concurrence avec le flux courant.

● Découper et espacer

Charger par tranches d'un chunk, avec une pause entre chaque, garde le flux courant dans son enveloppe de latence.

● Créer les index après

Sur une reprise d'historique importante, supprimer les index secondaires, charger, puis les recréer, divise la durée.

● Attention aux chunks déjà compressés

Écrire dans un chunk passé au columnstore est possible mais coûteux. Le sujet est traité en M09 ; ici, il suffit de savoir que le backfill peut y toucher.

Données tardives : où elles atterrissent

Elles vont dans le chunk de leur horodatage

- Pas dans le chunk courant. C'est le comportement attendu, et c'est ce qui rend l'historique correct.

Elles invalident les agrégats déjà matérialisés

- Un point arrivé après coup rend faux le seau qui le contient. L'invalidation est enregistrée, le rattrapage est différé — mécanisme détaillé en M08.

Elles peuvent viser un chunk compressé

- Selon leur retard et la politique de bascule. Le coût n'est plus le même : sujet de M09.

La question qui compte ce n'est pas « y a-t-il des données tardives », c'est « de combien sont-elles tardives ».

Mesurer le retard d'arrivée

Comparer l'horodatage de la mesure à celui de son insertion. C'est la seule façon d'obtenir un chiffre plutôt qu'une impression.

```
-- une colonne d'horodatage d'ingestion, alimentée par défaut
ALTER TABLE mesures
  ADD COLUMN ingere_le TIMESTAMPTZ NOT NULL DEFAULT now();

-- distribution du retard, sur les dernières 24 h
SELECT percentile_disc(0.50) WITHIN GROUP (ORDER BY ingere_le - ts) AS p50,
       percentile_disc(0.99) WITHIN GROUP (ORDER BY ingere_le - ts) AS p99,
       max(ingere_le - ts) AS pire
FROM   mesures WHERE ingere_le >= now() - interval '24 hours';
```

Pourquoi cette valeur voyage le décalage de la fenêtre de rafraîchissement d'un agrégat continu doit couvrir le retard réel. Le dimensionner au p99 plutôt qu'au p50 est une décision, et elle se prend en M08 avec ce chiffre.

Le pool surdimensionné

Deux cents connexions sur deux cœurs ne produisent pas deux cents fois le débit. Elles le réduisent.

Chaque connexion est un processus

- Mémoire, changements de contexte, contention sur les verrous internes. Au-delà du nombre de cœurs, on ajoute du travail, pas de la capacité.

Le point optimal se mesure

- Un ordre de grandeur usuel est de deux à quatre fois le nombre de cœurs. C'est un point de départ à vérifier, pas une formule.

Le pooling en mode transaction

- PgBouncer permet à de nombreux clients de partager peu de connexions serveur. C'est la réponse au besoin de concurrence côté application.



ATELIER L04

Montée en charge

Trois stratégies, un point de rupture, une ressource limitante nommée, et le retard d'arrivée mesuré.

Brief

SOCLE · pour tous

- Injecter pendant 60 secondes selon trois stratégies : unitaire, lots de 1 000, COPY binaire
- Mesurer pour chacune débit, latence médiane, CPU et volume de journal produit
- Calculer le rapport entre la stratégie naïve et la meilleure
- Monter jusqu'au point de rupture et nommer la ressource saturée, mesure à l'appui
- Mesurer le retard d'arrivée et le consigner : M08 en dépend

EXTENSION · pour aller plus loin

- Rendre l'ingestion idempotente, rejouer un lot, vérifier l'absence de doublon et mesurer le surcoût
- Charger 30 jours d'historique pendant que le flux courant tourne, et mesurer sa dégradation
- Comparer table journalisée et non journalisée : chiffrer le gain, énoncer le risque
- Faire varier la taille de lot de 100 à 50 000 et situer le plateau sur la courbe

Réussi si le rapport dépasse un ordre de grandeur, la ressource limitante est nommée sur la base d'une mesure, et le retard d'arrivée est consigné.

Le tableau à remplir

Grandeur relevée	Unitaire	Lots 1 000	COPY binaire
Débit · points par seconde			
Latence médiane par opération			
CPU pendant l'injection			
Journal produit			
Lignes écrites en 60 s			

Le CPU saturé et le disque saturé ne se corrigent pas de la même façon. Nommer la ressource, ce n'est pas dire « c'est lent ».

Pièges et indices

Le rapport mesuré ne sera pas transposable

Il dépend de la latence réseau. En local il sera bien plus faible qu'en production. Consigner le contexte à côté du chiffre.

Le volume de journal semble nul

Il se relève par différence de position, pas par la taille du répertoire, qui est recyclé en continu.

Le point de rupture n'arrive jamais

L'injecteur sature avant la base. Vérifier le CPU du client avant de conclure sur le serveur.

Le retard d'arrivée mesuré est nul

L'injecteur produit des horodatages à l'instant de l'insertion. Utiliser le générateur avec décalage prévu dans l'énoncé, sans quoi la valeur exportée vers M08 est inutilisable.

État attendu

Livrable du module

- Stratégie d'ingestion retenue et mesurée
- Retard d'arrivée : p50, p99, pire cas
- État de reprise mistral-M05

Carte des concepts

6 / 12

Ajout de la donnée tardive,
désormais chiffrée et non plus
seulement nommée.

Resynchronisation

- Le retard d'arrivée est la seule valeur de ce module dont M08 a besoin : la vérifier avant la pause
- Un participant sans point de rupture atteint restaure mistral-M05 et reprend l'injection avec l'injecteur de référence
- Un rapport inférieur à dix signale un injecteur saturé, pas une base performante

CE QUI SUIV

M06 · Interrogation temporelle

On sait écrire vite. Les tableaux de bord de MISTRAL mettent toujours quarante secondes.

1

Les seaux temporels

time_bucket, ses fuseaux et son piège le plus silencieux : l'agrégation journalière qui change d'heure.

2

Les trous et l'interpolation

Distinguer une absence de mesure d'une mesure nulle. Une question qui remonte systématiquement en industrie.

3

Réécrire les motifs coûteux

Cinq requêtes de la base MISTRAL initiale, réécrites, mesurées et expliquées par leur plan.

Le fil se resserre

Les cinq requêtes de M06 viennent de mistral_legacy — celle de l'amorce de M01. C'est là que l'écart de quarante secondes commence réellement à se combler.



MODULE M06

Interrogation temporelle

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M05

- Un ordre de grandeur mesuré entre stratégie naïve et COPY binaire
- Le point de rupture est atteint et la ressource limitante nommée
- Le retard d'arrivée est chiffré — M08 s'en servira

Ce qui manque encore

- Les tableaux de bord mettent toujours quarante secondes
- Le partitionnement n'y changera rien : le problème est dans l'écriture des requêtes
- Les courbes présentent des trous que personne n'a expliqués

Objectif du module écrire des agrégations temporelles justes, et réécrire les motifs coûteux en mesurant le gain.

Agenda et objectifs

- 1 Les seaux temporels
- 2 Trous, interpolation, et ce qui est faux
- 3 Motifs coûteux et réécriture
- L Atelier L05 · réécriture de cinq requêtes
- ✓ Point d'acquis

Deux fautes se ressemblent une requête lente se voit et se corrige. Une requête fausse retourne un chiffre, et personne ne la corrige.



BLOC 6.1

Les seaux temporels

Discrétiser l'axe du temps. La base de toute agrégation, et le lieu d'un piège silencieux.

time_bucket

```
-- largeur du seau, colonne de temps
SELECT time_bucket(INTERVAL '1 hour', ts) AS heure,
       avg(valeur), count(*)
FROM   mesures
WHERE  ts >= :'debut' AND ts < :'fin'
GROUP BY 1 ORDER BY 1;

-- decalage d'origine : des seaux alignes sur 6h05, pas sur l'heure ronde
SELECT time_bucket(INTERVAL '1 hour', ts, INTERVAL '5 minutes') ...
```

Sargable

Le prédicat reste sur ts brut : l'exclusion de chunks de M04 fonctionne.

Déterministe

Le même point tombe toujours dans le même seau, quel que soit l'ordre de lecture.

Composable

Un seau horaire s'agrège en seau journalier : c'est la base des hiérarchies de M08.

L'agrégation journalière et le fuseau

Un seau d'un jour calculé en UTC ne commence pas à minuit heure locale. Sur une production d'énergie facturée à la journée, le chiffre est faux toute l'année.

```
-- faux : les journees commencent a 02h00 heure locale en ete
SELECT time_bucket(INTERVAL '1 day', ts) ...

-- juste : les journees commencent a minuit heure locale, toute l'annee
SELECT time_bucket(INTERVAL '1 day', ts, 'Europe/Paris') ...
```

Et le week-end de changement d'heure

Deux jours par an durent 23 et 25 heures. Sans le fuseau, ces deux journées ont une durée fausse — en plus du décalage de deux heures qui fausse chaque journée de l'année (de -3% à $+5\%$ sur MISTRAL). L'écart n'apparaît sur aucun contrôle de cohérence : les totaux de période restent justes. C'est le mode de défaillance le plus silencieux du module.

Largeurs fixes, largeurs variables

Largeur fixe

Minute, heure, jour, semaine. Tous les seaux ont la même durée, l'arithmétique est simple et l'empilement immédiat.

C'est le cas usuel, et celui des hiérarchies de M08.

Largeur variable

Mois, trimestre, année. Les seaux n'ont pas la même durée : il faut la forme prenant un fuseau, faute de quoi la notion de mois n'est pas définie.

À réserver aux restitutions métier, pas aux niveaux intermédiaires.

```
-- comparaison periode sur periode : meme seau, deux fenetres
SELECT b, valeur, lag(valeur) OVER (ORDER BY b) AS periode_precedente
FROM ( SELECT time_bucket(INTERVAL '1 day', ts, 'Europe/Paris') AS b,
           sum(valeur)/360 AS valeur FROM mesures GROUP BY 1 ) x; -- kWh, pas un compteur
```



BLOC 6.2

Trous, interpolation, et ce qui est faux

Comblé un trou est un geste technique. Décider s'il faut le combler est une décision métier.

Absence de mesure, ou mesure nulle

La question remonte systématiquement en industrie, et les deux cas se ressemblent dans une courbe.

Absence de mesure

Le capteur n'a rien remonté : liaison coupée, machine à l'arrêt, collecteur en panne. On ne sait pas ce qui s'est passé.

Une éolienne dont la production est inconnue.

Mesure nulle

Le capteur a remonté zéro. On sait ce qui s'est passé, et la réponse est « rien ».

Une éolienne à l'arrêt qui produit zéro kilowatt.

Ce que coûte la confusion

Un taux de disponibilité calculé en traitant les absences comme des zéros compte une panne de collecteur comme une panne de machine. L'indicateur devient faux, et il est publié.

time_bucket_gapfill : générer les seaux absents

```
SELECT time_bucket_gapfill(INTERVAL '1 hour', ts) AS heure,  
       avg(valeur)                               AS brut,  
       locf(avg(valeur))                         AS derniere_connue,  
       interpolate(avg(valeur))                  AS interpolee  
FROM   mesures  
WHERE  series_id = 137  
      AND ts >= '2026-10-01' AND ts < '2026-10-08' -- bornes obligatoires  
GROUP BY 1 ORDER BY 1;
```

Les bornes sont obligatoires

Sans elles, la fonction ne sait pas quels seaux engendrer.
L'erreur retournée est déroutante.

La colonne brute reste NULL

gapfill crée la ligne ; locf et interpolate remplissent. Conserver
les trois colonnes côte à côte rend l'écart visible.

La nature du signal décide, pas la commodité

Signal MISTRAL	LOCF	Interpolation	Pourquoi
Compteur d'énergie	légitime	légitime	Grandeur cumulée, monotone entre deux relevés
Vitesse du vent	faute	faute	Grandeur volatile : toute valeur inventée est fausse
Consigne d'angle de pale	légitime	faute	Valeur en escalier : elle tient jusqu'au changement
État de la machine	légitime	absurde	Grandeur discrète : interpoler entre deux états n'a pas de sens
Température de nacelle	acceptable	légitime	Grandeur continue et lente

Aucune de ces cases ne se déduit du type SQL de la colonne. Elles se déduisent du capteur — c'est-à-dire du référentiel.

Quand combler est une faute

Le trou est l'information

- Une interruption de collecte est un incident à détecter, pas un défaut d'affichage à corriger. Le combler supprime le signal d'alerte.

Le résultat sert à facturer ou à certifier

- Une valeur inventée qui traverse une chaîne de facturation ne se distingue plus d'une valeur mesurée. Conserver la distinction jusqu'au bout.

Le trou est plus long que la dynamique du signal

- LOCF sur six heures de vent absent produit une droite qui n'a jamais existé. Fixer une durée maximale de comblement, et la documenter.

Une bonne réponse à « comment combler ce trou » est parfois « ne le comblez pas, et affichez pourquoi il est là ».



BLOC 6.3

Motifs coûteux et réécriture

Le motif le plus fréquent en production est aussi le plus mal écrit.

« Dernière valeur de chaque capteur »

Le motif présent dans tous les tableaux de bord, et l'écriture qu'on trouve dans toutes les bases.

L'écriture naïve : huit secondes sur MISTRAL

```
SELECT m.series_id, m.ts, m.valeur
FROM   mesures m
JOIN   ( SELECT series_id, max(ts) AS ts
        FROM   mesures GROUP BY series_id ) d
      ON d.series_id = m.series_id AND d.ts = m.ts;
```

Pourquoi c'est coûteux

La sous-requête agrège l'intégralité de l'hypertable pour ne retenir qu'une ligne par série. Aucun prédicat temporel, donc aucune exclusion de chunks : les 190 millions de lignes sont lues, puis la jointure les relit.

Trois réécritures

DISTINCT ON

```
SELECT DISTINCT ON (series_id)
       series_id, ts, valeur
FROM   mesures
ORDER  BY series_id, ts DESC;
```

Jointure latérale

```
SELECT s.series_id, m.ts, m.valeur
FROM   signaux s
CROSS JOIN LATERAL (
    SELECT ts, valeur FROM mesures
    WHERE  series_id = s.signal_id
    ORDER  BY ts DESC LIMIT 1) m;
```

Et la troisième : laisser l'index faire le travail

Avec un index sur (series_id, ts DESC), le moteur peut parcourir l'index en sautant d'une série à la suivante sans lire les lignes intermédiaires. Le plan ne ressemble alors plus du tout à un balayage : c'est le même SQL, avec un index en face, et l'écart se mesure en ordres de grandeur.

Laquelle gagne, et à quelle condition

Écriture	Gagne quand	Perd quand
Sous-requête + jointure	Jamais, sur ce motif	Toujours : deux balayages complets
DISTINCT ON	Peu de séries, index adapté	Le nombre de séries devient élevé
Jointure latérale	Le référentiel des séries est petit et connu	Le référentiel contient des séries mortes : autant de recherches inutiles
Index et parcours par saut	Beaucoup de séries, index sur (série, temps)	L'index manque, ou l'ordre de ses colonnes est inversé

Le seuil entre DISTINCT ON et le parcours par saut se mesure sur les données réelles.

Deux outils qui évitent une sous-requête

first et last dans une agrégation

```
-- premiere et derniere valeur de chaque seau, sans sous-requete
SELECT time_bucket(INTERVAL '1 hour', ts) AS heure, series_id,
       first(valeur, ts) AS ouverture,
       last(valeur, ts) AS cloture
FROM   mesures GROUP BY 1, 2;
```

Fenêtrage sur données temporelles

```
-- ecart avec le seau precedent, par serie
SELECT heure, series_id, moyenne,
       moyenne - lag(moyenne) OVER (PARTITION BY series_id ORDER BY heure)
FROM   ( ... ) x;
```



ATELIER L05

Réécriture de cinq requêtes

Cinq requêtes extraites de `mistral_legacy`, telles qu'elles tournent aujourd'hui en production.

Brief

SOCLE · pour tous

- Réécrire les cinq requêtes sur le schéma hypertable
- Mesurer avant et après, produire le plan, identifier le nœud responsable du gain
- Rendre le pas de la requête 3 régulier, et justifier le traitement retenu — y compris ne rien remplir
- Faire passer la requête 5 — dernière valeur par capteur — sous la seconde
- Consigner les cinq gains dans mesures.md

EXTENSION · pour aller plus loin

- Reproduire le bug de fuseau sur une semaine de changement d'heure, et chiffrer l'écart en énergie
- Écrire trois variantes de « dernière valeur par série » et les classer par mesure, pas par élégance
- Traiter le cas où l'absence doit rester visible et où gapfill serait une faute métier
- Comparer largeur fixe et largeur variable sur des seaux mensuels

Réussi si les cinq requêtes sont réécrites, chaque gain est mesuré, et le participant explique le plan de la requête 5.

Les cinq requêtes

R1

Production horaire par site, sur trente jours

Seau temporel et jointure au référentiel

R2

Énergie journalière facturable

Fuseau horaire, et somme d'un compteur cumulatif — la requête est fausse, pas lente

R3

Courbe de vent à pas régulier

Trous : gapfill, et le choix LOCF ou interpolation

R4

Comparaison d'un mois avec le mois précédent

Seaux de largeur variable et fenêtrage — l'énergie se dérive de la puissance, jamais d'une somme de compteur

R5

Dernière valeur de chaque capteur

Le motif du bloc 6.3 — huit secondes à faire tomber sous la seconde

R2 est la seule des cinq qui ne soit pas lente. Elle est fausse — et c'est la plus difficile à repérer.

Pièges et indices

gapfill retourne une erreur déroutante

- Les bornes temporelles doivent figurer explicitement dans le WHERE, ou être passées à la fonction. Sans elles, elle ne sait pas quels seaux engendrer.

R2 semble correcte

- Elle l'est sur le total de la période. Comparer jour par jour : la fenêtre décalée de deux heures fausse chaque journée, et le jour de bascule ajoute une durée fausse de 23 ou 25 h.

La réécriture de R5 n'apporte rien

- Vérifier l'index. Sans (series_id, ts DESC), aucune des trois réécritures ne peut gagner — c'est l'index qui fait le travail, pas la syntaxe.

Le gain de R1 est modeste

- Elle porte déjà un prédicat temporel : l'exclusion jouait déjà. Le gain viendra de M08, et c'est une information, pas un échec.

État attendu

Livrable du module

- requetes.sql : cinq réécritures commentées
- Cinq gains mesurés dans mesures.md
- État de reprise mistral-M06

Carte des concepts

7 / 12

Ajout du seau temporel, désormais instancié sur les cinq requêtes.

Resynchronisation

- R5 est la seule des cinq dont la réécriture est reprise dans les modules suivants : la vérifier avant la pause
- Un participant en retard restaure mistral-M06 et garde ses propres réécritures dans son plan d'application
- Un gain nul sur R1 est un résultat correct, pas un échec : le dire pour éviter la conclusion inverse

M07 • Hyperfonctions et agréabilité

Les requêtes sont rapides. Reste à savoir si leurs résultats sont justes.

1

La moyenne qui ment

Sur un pas d'échantillonnage irrégulier, la moyenne arithmétique donne un chiffre faux — et personne ne le voit.

2

Compteurs et états

Une remise à zéro fausse un cumul annuel. Un temps passé dans un état ne se calcule pas comme une moyenne.

3

Ce qui remonte une hiérarchie

La notion qui commande tout M08 : une moyenne de moyennes est fausse, un percentile ne s'agrège pas.

Le module le plus court, et le plus structurant

Sans lui, les hiérarchies d'agrégats de M08 s'enseignent sur du dogme : on saurait empiler des niveaux sans savoir lesquels ont le droit de l'être.



MODULE M07

Hyperfonctions et agréabilité

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M06

- Cinq requêtes réécrites, cinq gains mesurés
- Une requête fausse identifiée et corrigée : le fuseau de R2
- Le trou laissé visible quand le remplir serait une faute

Ce qui manque encore

- R1 et R4 calculent des sommes — des grandeurs qui se comportent bien
- Trois indicateurs métier de MISTRAL ne sont ni des sommes ni des comptages
- Personne ne sait encore lesquels peuvent être empilés en hiérarchie

Objectif du module savoir quels indicateurs remontent une hiérarchie, et lesquels imposent un état intermédiaire.

Agenda et objectifs

1 Les indicateurs qui ne se calculent pas comme des sommes

2 Réagrégation et états intermédiaires

L Atelier L06 · trois indicateurs MISTRAL

✓ Point d'acquis

Le module le plus court, et celui sans lequel M08 s'enseigne sur du dogme

Empiler des niveaux d'agrégats est facile. Savoir lesquels ont le droit de l'être suppose de connaître la notion d'agrégabilité : c'est le seul objet de ce module.



BLOC 7.1

Les indicateurs qui ne se calculent pas comme des sommes

Trois grandeurs de MISTRAL, et trois façons différentes de se tromper.

La moyenne arithmétique sur un pas irrégulier

Une éolienne remonte sa puissance toutes les dix secondes, sauf pendant l'arrêt de maintenance où elle remonte une fois par heure.

avg(valeur)

Chaque point compte pour un, quelle que soit la durée qu'il représente. Les rares points de la période d'arrêt pèsent autant que les milliers de points de la période nominale.

Résultat : la moyenne est tirée vers le haut, et la production paraît meilleure qu'elle ne l'a été.

Moyenne pondérée par le temps

Chaque point compte pour la durée qu'il couvre. Les points de la période d'arrêt pèsent une heure chacun.

Résultat : la grandeur reflète ce qui s'est réellement passé sur l'axe du temps.

Le pas irrégulier n'est pas une anomalie du jeu de données : c'est la situation normale de toute télémétrie industrielle.

Pondérer par le temps

```
SELECT time_bucket(INTERVAL '1 hour', ts) AS heure,  
       series_id,  
       average(time_weight('LOCF', ts, valeur)) AS moyenne_ponderee,  
       avg(valeur)                               AS moyenne_naive  
FROM   mesures  
WHERE  ts >= :'debut' AND ts < :'fin'  
GROUP BY 1, 2;
```

Le paramètre de méthode

LOCF suppose que la valeur tient jusqu'au point suivant. La méthode linéaire suppose une variation continue entre deux points. Le même choix qu'en M06, et les mêmes critères.

Ce que la fonction retourne

Un objet intermédiaire, pas un nombre. C'est `average()` qui en extrait la moyenne — et c'est cette dissociation qui rend la réagrégation possible.

Compteurs : la remise à zéro qui efface une année

Un compteur d'énergie est monotone croissant jusqu'au remplacement de la carte, où il repart de zéro.

L'écriture naïve, et ce qu'elle produit après un redémarrage

```
SELECT max(compteur) - min(compteur) AS energie_periode ...  
-- apres une remise a zero : resultat negatif, ou fortement sous-estime
```

L'agrégat de compteur, qui connaît la notion de redémarrage

```
SELECT time_bucket(INTERVAL '1 day', ts, 'Europe/Paris') AS jour,  
       delta(counter_agg(ts, compteur)) AS energie_jour,  
       rate(counter_agg(ts, compteur)) AS puissance_moyenne  
FROM   mesures_production GROUP BY 1;
```

La fonction détecte la discontinuité et recolle les segments. C'est une connaissance métier encodée dans l'agrégat, pas une astuce de calcul.

États : le temps passé, et la disponibilité

Une machine passe par une succession d'états datés. La question métier n'est pas « combien de fois », c'est « combien de temps ».

```
SELECT machine_id,  
       duration_in(state_agg(ts, etat), 'production') AS temps_production,  
       duration_in(state_agg(ts, etat), 'arret')      AS temps_arret,  
       duration_in(state_agg(ts, etat), 'maintenance') AS temps_maintenance  
FROM   evenements  
WHERE  ts >= : 'debut' AND ts < : 'fin'  
GROUP BY 1;
```

Le piège du taux de disponibilité

Le dénominateur n'est pas la durée de la fenêtre, mais la durée pour laquelle on dispose d'une information d'état. Confondre les deux compte une panne de collecteur comme une panne de machine — le même piège qu'au bloc 6.2, sur un autre indicateur.



BLOC 7.2

Réagrégation et états intermédiaires

La notion qui commande toute l'architecture des agrégats de M08.

Ce qui remonte, et ce qui ne remonte pas

Indicateur	Se réagrège ?	Ce qu'il faut conserver au niveau bas
Somme, minimum, maximum	oui	La valeur suffit
Comptage	oui	Le compte suffit
Moyenne simple	non	La somme et le compte, pas la moyenne
Moyenne pondérée par le temps	conditionnel	L'objet intermédiaire complet
Percentile, médiane	non	Une structure d'approximation réagrégeable
Écart-type, variance	non	Les moments d'ordre 1 et 2, et le compte

Une moyenne de moyennes horaires n'est pas la moyenne journalière, sauf si toutes les heures ont le même nombre de points. Ce n'est jamais le cas.

Stocker l'état, pas le résultat

La solution tient en une inversion : au niveau bas, on ne conserve pas la valeur calculée, on conserve de quoi la recalculer plus haut.

```
-- niveau horaire : on stocke l'objet intermediaire
SELECT time_bucket(INTERVAL '1 hour', ts) AS heure, series_id,
       percentile_agg(valeur) AS etat
FROM   mesures GROUP BY 1, 2;

-- niveau journalier : on reagrege les objets, pas les resultats
SELECT time_bucket(INTERVAL '1 day', heure) AS jour, series_id,
       approx_percentile(0.95, rollup(etat)) AS p95_journalier
FROM   agregat_horaire GROUP BY 1, 2;
```

À retenir pour M08 un niveau d'agrégat qui stocke des moyennes est un cul-de-sac. Un niveau qui stocke des états intermédiaires se réagrége indéfiniment.

Percentiles : précision contre agrégabilité

Percentile exact

Impose de trier l'ensemble des valeurs. Le résultat est juste, il n'est pas réagrégeable, et le coût croît avec le volume.

À réserver aux calculs ponctuels sur une fenêtre bornée.

Percentile approché

Une structure de résumé bornée en taille, réagrégeable, avec une erreur maîtrisée et documentée.

C'est ce qui rend un percentile utilisable dans une hiérarchie d'agrégats.

Ce qu'il faut dire à un métier qui demande « le vrai p95 »

L'approximation a une erreur bornée et connue. Le percentile exact recalculé chaque nuit sur des données incomplètes en a une aussi — mais celle-là n'est ni bornée ni mesurée.

Où ces fonctions sont disponibles

Elles relèvent du Toolkit, sous licence TSL

- Comme le columnstore et les agrégats continus. La licence n'interdit pas l'usage en production ; elle interdit de revendre le produit comme service.

Le Toolkit s'installe séparément de l'extension

- Une instance peut avoir TimescaleDB sans le Toolkit. C'est un cas fréquent sur les plateformes managées tierces, et il faut le vérifier avant de concevoir.

Sans Toolkit, tout reste faisable — autrement

- Moyenne pondérée, delta de compteur et durée d'état s'écrivent en SQL avec des fonctions de fenêtrage. C'est plus long, plus lent, et il faut gérer les cas limites soi-même.

Vérifier la présence du Toolkit fait partie de la mise en service. C'est une ligne à ajouter à la checklist de M02.



ATELIER L06

Trois indicateurs MISTRAL

Production pondérée, taux de disponibilité, percentile de vent. Aucun des trois n'est une somme.

Brief

SOCLE · pour tous

- Production mensuelle pondérée par le temps, par machine
- Taux de disponibilité par machine, avec le bon dénominateur
- Percentile 95 de la vitesse du vent, par site
- Pour le premier : écrire aussi la version naïve, et expliquer pourquoi elle est fausse

EXTENSION · pour aller plus loin

- Démontrer qu'une moyenne de moyennes horaires diverge de la moyenne journalière, et chiffrer l'écart
- Stocker un état de percentile au niveau horaire, le réagréger au jour, comparer au calcul direct
- Réduire une série de 8 640 points à 500 en conservant la forme de la courbe
- Réécrire un des trois indicateurs sans le Toolkit, et mesurer l'écart de durée

Réussi si les trois indicateurs sont corrects, et l'erreur de la version naïve est expliquée — pas récitée.

Pièges et indices

Le Toolkit n'est pas installé

- Vérifier avant de commencer. Sans lui, aucune des trois fonctions n'existe, et le message d'erreur ne suggère pas l'installation.

Le taux de disponibilité dépasse cent pour cent

- Le dénominateur retenu est la durée de la fenêtre alors que les états ne la couvrent pas entièrement. Reprendre le piège du bloc 7.1.

La version naïve donne presque le même résultat

- C'est possible si le pas est régulier sur la fenêtre choisie. Choisir une machine et un mois comportant un arrêt de maintenance → la liste est dans l'énoncé.

Le percentile approché et le percentile exact diffèrent

- C'est attendu, et l'écart doit être borné. Le mesurer plutôt que de le constater : c'est ce qui rend l'approximation défendable devant un métier.

État attendu

Livrable du module

- Trois indicateurs métier corrects
- La version naïve réfutée par écrit
- État de reprise mistral-M07

Carte des concepts

8 / 12

Ajout de l'agrégabilité : la notion qui commande M08.

Resynchronisation

- Un participant qui ne sait pas dire pourquoi une moyenne de moyennes est fausse n'est pas prêt pour M08
- C'est la seule notion du module qui soit strictement bloquante → les trois fonctions, elles, se retrouvent dans la documentation
- La question de contrôle se pose à voix haute, en trente secondes, avant la pause

M08 · Agrégats continus

Les requêtes sont rapides et justes. Elles restent recalculées à chaque affichage.

1

La matérialisation incrémentale

Ce qu'une vue matérialisée PostgreSQL ne sait pas faire : ne recalculer que ce qui a changé.

2

La fenêtre de rafraîchissement

Dimensionnée avec le retard d'arrivée mesuré en M05. Deux valeurs produites deux modules plus tôt se rejoignent ici.

3

La pyramide

Une minute, une heure, un jour — et seulement des indicateurs qui ont le droit d'y monter.

Deux modules convergent

M05 a produit le retard d'arrivée, M07 vient de produire la règle d'agrégabilité. M08 est le premier module qui ne peut se passer d'aucun des deux.



MODULE M08

Agrégats continus

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M05 et M07

- Le retard d'arrivée est chiffré : p50, p95, p99, pire cas
- La règle d'agrégabilité est posée, et le tableau du bloc 7.2 est écrit
- Trois indicateurs métier corrects, et recalculés à chaque exécution

Ce qui manque encore

- Chaque affichage relit les mesures brutes : rien n'est matérialisé
- Aucune décision sur ce que le niveau horaire doit stocker
- Personne ne sait ce qui se passe quand un point arrive après coup

Le premier module qui ne peut se passer d'aucun des deux le retard dimensionne les fenêtres, l'agrégabilité décide du contenu des niveaux.

Agenda et objectifs

- 1 Matérialisation incrémentale
- 2 La fenêtre de rafraîchissement
- 3 Pyramide et évolution en production
- L Atelier L07 · pyramide d'agrégats MISTRAL
- ✓ Point d'acquis

La faute à ne pas commettre un niveau d'agrégat qui stocke des moyennes est un cul-de-sac. Le tableau du bloc 7.2 est à portée de main.



BLOC 8.1

Matérialisation incrémentale

Ce qu'une vue matérialisée PostgreSQL ne sait pas faire.

Vue matérialisée, ou agrégat continu

Vue matérialisée PostgreSQL

- Le rafraîchissement recalcule la totalité
- Sur 190 millions de lignes, le coût est le même chaque nuit
- Aucune notion de ce qui a changé depuis la dernière fois

Agrégat continu

- Seules les périodes modifiées sont recalculées
- Le coût est proportionnel au changement, pas au volume
- Les modifications sont tracées au fil de l'eau

Ce que MISTRAL fait aujourd'hui des vues matérialisées rafraîchies par cron, sans suivi d'invalidation. C'est la pathologie n° 3 de M01.

Créer un agrégat continu

```
CREATE MATERIALIZED VIEW mesures_horaire
WITH (timescaledb.continuous) AS
SELECT time_bucket(INTERVAL '1 hour', ts) AS heure,
       series_id,
       avg(valeur)    AS moyenne,
       count(*)       AS points,
       sum(valeur)    AS somme      -- pour pouvoir reagreger
FROM   mesures
GROUP BY 1, 2
WITH NO DATA;                  -- materialiser ensuite, pas maintenant
```

Pourquoi somme et compte, et pas seulement la moyenne parce qu'une moyenne ne se réagrège pas. Le niveau journalier aura besoin de la somme et du compte. C'est la règle de M07, appliquée dès la définition.

Le suivi d'invalidation

1

Une écriture touche une période déjà matérialisée

Insertion tardive, correction, suppression : peu importe l'opération.

2

La période concernée est enregistrée

Pas la ligne, la période. Le mécanisme raisonne en intervalles de temps.

3

Le rafraîchissement suivant traite ces périodes

Il ne recalcule que les seaux marqués, plus ceux de sa fenêtre courante.

4

Entre les deux, l'agrégat est en retard

C'est un choix, pas un défaut : la correction est différée pour ne pas payer à l'écriture.

Ce retard assumé est exactement ce que la fenêtre de rafraîchissement du bloc suivant doit dimensionner.

Agrégation en temps réel : matérialisé plus frais

Par défaut, une lecture combine ce qui est matérialisé et ce qui ne l'est pas encore. Le résultat est à jour — et la lecture coûte davantage.

```
-- desactiver : la vue ne retourne que ce qui est materialise
ALTER MATERIALIZED VIEW mesures_horaire
SET (timescaledb.materialized_only = true);
```

Garder le temps réel

Le tableau de bord affiche la dernière heure en cours.
Chaque lecture ajoute un balayage des données fraîches.

Le bon choix quand la fraîcheur prime et que la vue est peu sollicitée.

Ne lire que le matérialisé

La vue ne retourne rien au-delà de sa dernière matérialisation. La lecture devient très rapide et parfaitement prévisible.

Le bon choix sur une vue très sollicitée, ou sur un niveau haut d'une pyramide.

Ce que la définition autorise, et ce qu'elle refuse

Autorisé

- time_bucket sur la colonne de temps de l'hypertable
- Les agrégats usuels et ceux du Toolkit
- Un GROUP BY contenant le seuil
- Les jointures, sous conditions

Refusé

- ORDER BY, LIMIT, DISTINCT
- Les fonctions de fenêtrage
- Une définition sans time_bucket
- Les agrégats non parallélisables

La conséquence pratique un classement ou un décalage période sur période se calcule à la lecture, au-dessus de l'agrégat, jamais dans sa définition. La liste précise évolue avec les versions : la vérifier plutôt que la mémoriser.



BLOC 8.2

La fenêtre de rafraîchissement

Trois paramètres, et une valeur mesurée deux modules plus tôt.

Trois paramètres

```
SELECT add_continuous_aggregate_policy('mesures_horaire',  
  start_offset      => INTERVAL '3 days',    -- borne ancienne  
  end_offset        => INTERVAL '1 hour',    -- borne recente  
  schedule_interval => INTERVAL '30 minutes' -- frequence  
);
```

start_offset

Jusqu'où l'on remonte à chaque passage. Doit couvrir la fenêtre pendant laquelle des données tardives peuvent encore arriver.

end_offset

Jusqu'où l'on s'arrête, en s'écartant du présent. Matérialiser un seau encore en cours de remplissage produit une valeur partielle qui sera figée.

schedule_interval

À quelle fréquence le travail s'exécute. Trop court, il tourne à vide ; trop long, l'agrégat est durablement en retard.

Dimensionner avec le retard d'arrivée

Le p50, le p95, le p99 et le pire cas ont été mesurés en M05. Chacun conduit à une décision différente, et à une perte différente.

start_offset retenu	Ce qu'on gagne	Ce qu'on accepte de perdre
Le p50	Un travail très court à chaque passage	La moitié des points tardifs ne seront jamais repris
Le p95	Un bon compromis coût / justesse	Cinq pour cent des points tardifs sont perdus
Le p99	Presque tout est repris	Le travail remonte plus loin, donc coûte plus cher à chaque passage
Le pire cas	Rien n'est perdu	Chaque passage recalcule une fenêtre très large, en pure perte la plupart du temps

Il n'y a pas de bonne réponse universelle. Il y a une décision à prendre, et un chiffre pour la prendre.

Fréquence contre coût, et rattrapage manuel

Le travail périodique

Il s'exécute sur l'ordonnanceur intégré, comme toutes les politiques. Son historique d'exécution est consultable, et c'est là qu'on voit qu'il échoue.

Sujet approfondi en M11.

Le rattrapage manuel

Après un backfill ou une correction massive, on force la matérialisation d'une période précise sans attendre le passage suivant.

À faire hors des heures de charge : c'est un balayage complet de la période visée.

```
CALL refresh_continuous_aggregate('mesures_horaire',  
    '2026-10-01', '2026-10-08');  
-- ne peut pas s'exécuter dans une transaction explicite
```

Un appel manuel sur une fenêtre trop large est le moyen le plus simple de saturer une instance en production.



BLOC 8.3

Pyramide et évolution en production

Empiler les niveaux, et vivre avec une définition qu'on ne peut pas modifier.

Empiler les niveaux

```
-- le niveau journalier se definit AU-DESSUS du niveau horaire
CREATE MATERIALIZED VIEW mesures_jour
WITH (timescaledb.continuous) AS
SELECT time_bucket(INTERVAL '1 day', heure, 'Europe/Paris') AS jour,
       series_id,
       sum(somme) / sum(points) AS moyenne,    -- et non avg(moyenne)
       sum(points)              AS points,
       sum(somme)               AS somme
FROM   mesures_horaire
GROUP BY 1, 2 WITH NO DATA;
```

La ligne qui compte `sum(somme) / sum(points)`, et non `avg(moyenne)`. C'est toute la règle d'agrégabilité de M07, en une expression, et c'est la faute la plus fréquente sur une pyramide d'agrégats.

BLOC 8.3

Ce que le niveau horaire doit stocker

La décision d'architecture du module. Elle se lit directement dans le tableau du bloc 7.2.

Indicateur MISTRAL	Stocké au niveau horaire	Calculé au niveau journalier
Énergie produite	<code>sum(energie)</code>	<code>sum(sum)</code>
Puissance moyenne	<code>sum et count</code>	<code>sum(sum) / sum(count)</code>
Puissance maximale	<code>max</code>	<code>max(max)</code>
Percentile 95 du vent	<code>l'état de percentile</code>	<code>approx_percentile</code> sur le <code>rollup</code>
Taux de disponibilité	<code>l'état d'agrégat d'états</code>	<code>rollup</code> , puis <code>duration_in</code>

Aucune de ces lignes ne stocke le résultat au niveau bas. Toutes stockent de quoi le recalculer plus haut.

Ce que la pyramide économise

Journalier calculé sur le brut

Chaque rafraîchissement relit les mesures de la période concernée — plusieurs millions de lignes par jour et par série.

Journalier calculé sur l'horaire

Chaque rafraîchissement relit vingt-quatre lignes par série. Le facteur d'économie est celui du rapport entre les deux niveaux.

Le revers, à énoncer aussi

- Chaque niveau ajoute un délai : le journalier ne peut pas être plus frais que l'horaire dont il dépend
- Une donnée tardive doit remonter toute la pyramide, niveau par niveau → l'atelier le vérifie
- Trois niveaux, c'est trois politiques à surveiller, et trois occasions de silence en cas d'échec
- Un niveau intermédiaire que personne ne lit reste un coût de stockage et de calcul

Faire évoluer un agrégat déjà en production

Ce qui se modifie sans rien perdre

- Les paramètres de la politique, l'activation du temps réel, la compression et la rétention de l'agrégat lui-même.

Ce qui impose une recreation

- Toute modification de la requête : ajouter une colonne, changer la largeur du seau, modifier un GROUP BY. L'historique matérialisé est alors à reconstruire.

Comment on déploie malgré tout

- Créer le nouvel agrégat à côté sous un autre nom, le matérialiser sur l'historique, basculer les lecteurs, supprimer l'ancien. C'est une migration, avec sa fenêtre et son repli.

D'où la nécessité de versionner la définition au même endroit que le schéma, et de la déployer par le même chemin.



ATELIER L07

Pyramide d'agrégats MISTRAL

Trois niveaux, trois politiques, et une donnée tardive qui doit remonter jusqu'en haut.

Brief

SOCLE • pour tous

- Construire trois niveaux : une minute, une heure, un jour, chacun au-dessus du précédent
- Attacher une politique à chaque niveau, avec un décalage justifié par le retard mesuré en M05
- Rejouer les requêtes du tableau de bord et mesurer le gain
- Injecter des données tardives sur J-3, observer l'invalidation, déclencher le rattrapage
- Vérifier la cohérence aux trois niveaux par un contrôle de somme

EXTENSION • pour aller plus loin

- Comparer le coût d'une pyramide à celui d'un journalier calculé directement sur le brut
- Désactiver le temps réel et mesurer l'écart sur un tableau de bord sollicité
- Modifier la définition du niveau horaire et la déployer sans perdre l'historique
- Stocker un état de percentile au niveau horaire et le réagréger au jour

Réussi si le tableau de bord passe sous la seconde, et la donnée tardive est reprise aux trois niveaux — vérifié par un contrôle de somme, pas à l'œil.

Le protocole de la donnée tardive

1

Relever la somme aux trois niveaux sur J-3

Avant toute injection. C'est la référence du contrôle.

2

Injecter un lot daté de J-3

Cent mille points, sur des séries connues.

3

Constater que les trois niveaux n'ont pas bougé

C'est normal : l'invalidation est enregistrée, le rattrapage est différé.

4

Déclencher le rattrapage du niveau le plus bas

Puis constater que les niveaux supérieurs ne suivent pas seuls dès que la donnée tardive est plus ancienne que leur `start_offset` : les rattraper à la main, niveau par niveau, sur une fenêtre bornée.

5

Recontrôler les sommes

L'écart entre les trois niveaux doit être nul. S'il ne l'est pas, un niveau n'a pas été rattrapé.

L'étape 4 est celle qui surprend : la propagation entre niveaux n'est pas automatique. Un `run_job` forcé sur le niveau supérieur ne suffit pas si la période corrigée est hors de sa fenêtre de rafraîchissement — la propagation se fait à la main, bornée, du bas vers le haut.

Pièges et indices

Le décalage est trop court

Des seaux incomplets sont matérialisés puis figés. Personne ne le voit : les valeurs sont plausibles, simplement trop basses.

Le niveau supérieur refuse d'être créé

Les largeurs de seau doivent s'aligner : le jour doit être un multiple exact de l'heure, seuil de bascule compris.

avg(moyenne) au lieu de sum/sum

La faute la plus fréquente de l'atelier. Le résultat est plausible et faux — comparer au calcul direct sur le brut pour la révéler.

Le temps réel fausse la comparaison

Un niveau en temps réel et un autre non ne se comparent pas. Fixer le même réglage sur les trois avant le contrôle de somme.

État attendu

Livrable du module

- agregats.sql : trois niveaux avec politiques
- Gain du tableau de bord mesuré
- État de reprise mistral-M08

Carte des concepts

9 / 12

Ajout de l'agrégat continu, instancié sur trois niveaux.

Resynchronisation

- Le contrôle de somme aux trois niveaux est la seule preuve que la pyramide est juste : le vérifier avant la pause
- Un participant dont le décalage diffère de la référence garde son choix et le documente dans son plan d'application
- Une pyramide qui affiche des valeurs plausibles n'est pas une pyramide vérifiée

M09 • Hypercore et columnstore

Les requêtes sont rapides, justes et matérialisées. Il reste quinze gigaoctets pour quarante-cinq jours, et la facture double chaque année.

1

Deux états d'un même chunk

Rowstore et columnstore ne sont pas deux tables. Ce qui se passe à la bascule, et ce que cela change aux requêtes.



2

Segmenter et trier

Le choix qui divise le ratio par deux s'il est mal fait — davantage selon la cardinalité et la largeur des chunks — et qui dépend de la clé de série tranchée en M03.



3

La cardinalité, enfin démontrée

Un second jeu de données à vingt mille séries entre en scène, uniquement pour cette démonstration.

Le lab décisif de la formation

Le délai de bascule vers le columnstore doit dépasser la fenêtre de données tardives mesurée en M05 — la même valeur, pour la troisième fois.



MODULE M09

Hypercore et columnstore

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M08

- Trois niveaux d'agrégats, avec leurs politiques justifiées
- Le tableau de bord passe sous la seconde
- Une donnée tardive remonte la pyramide, contrôle de somme à l'appui

Ce qui manque encore

- Quinze gigaoctets pour quarante-cinq jours — et la pyramide en a ajouté
- Aucune donnée n'a jamais quitté le format ligne
- La cardinalité a été nommée en M01, jamais démontrée

Objectif du module choisir une segmentation sur la base d'une mesure, et chiffrer l'effet de la cardinalité sur le ratio.

Agenda et objectifs

1 Le modèle hypercore

2 Segmentation, tri, cardinalité

L Atelier L08 · segmentation comparée

✓ Point d'acquis

Le bloc le plus long de la formation, et le lab le plus décisif

Un segmentby mal choisi divise le ratio de compression par deux sur MISTRAL, rend les requêtes filtrées plus lentes qu'avant compression, et peut faire échouer une simple correction : sans segmentby, un UPDATE ciblé bute sur la limite de décompression. Aucune autre décision de la formation n'a un tel écart entre le bon et le mauvais choix.



BLOC 9.1

Le modèle hypercore

Rowstore et columnstore ne sont pas deux tables. Ce sont deux états du même chunk.

Deux états d'un même chunk

Rowstore

Le chunk fraîchement écrit. Une ligne = un enregistrement.
Écriture rapide, lecture ponctuelle rapide, volume maximal.

L'état des données chaudes.

Columnstore

Le même chunk, réorganisé par colonnes et par lots.
Volume divisé par un facteur important, scans analytiques accélérés.

L'état des données tièdes et froides.

Ce que « deux états » implique concrètement

Une seule hypertexte, un seul nom de table, une seule requête. Les chunks des deux états coexistent, et le moteur les lit ensemble sans que l'application le sache. Il n'y a pas de table d'archive à joindre, ni de vue d'union à maintenir.

Ce qui se passe physiquement à la bascule

1

Les lignes sont regroupées en lots

Un lot rassemble un millier de lignes environ, toutes issues du même segment.

2

Chaque colonne est stockée séparément

Une seule valeur de colonne par lot, dans un tableau compressé. Les colonnes homogènes compressent très bien.

3

Des métadonnées de lot sont calculées

Minimum, maximum et autres bornes par lot. Ce sont elles qui permettront d'écarter des lots entiers sans les lire.

4

Le chunk change d'état dans le catalogue

Il reste le même chunk, avec le même nom et les mêmes bornes temporelles.

Le regroupement en lots par segment est la clé de tout le module. C'est lui que segmentby pilote.

Activer et automatiser

```
ALTER TABLE mesures SET (  
    timescaledb.compress,  
    timescaledb.compress_segmentby = 'series_id',  
    timescaledb.compress_orderby   = 'ts DESC'  
);  
  
SELECT add_compression_policy('mesures', INTERVAL '7 days');  
  
-- le resultat obtenu, chunk par chunk  
SELECT * FROM hypertable_compression_stats('mesures');
```

Note de vocabulaire le nom « compression » est historique ; « columnstore » est le vocabulaire du modèle hypercore. Selon la version, les deux familles de noms coexistent pour les mêmes opérations. Vérifier laquelle est en usage sur l'instance cible avant d'écrire un script.

Effet sur les requêtes, profil par profil

Profil de requête	Effet	Pourquoi
Scan analytique large	fortement accéléré	Moins de blocs à lire, et seules les colonnes utiles sont décompressées
Agrégation sur une colonne	fortement accéléré	Le format colonne sert exactement ce profil
Point unique par série et instant	dégradé	Il faut décompresser le lot entier pour extraire une ligne
Dernière valeur par série	variable	Dépend entièrement de orderby — c'est l'objet du bloc suivant
Écriture dans la période	coûteux	Voir le slide sur l'écriture en columnstore

Le columnstore n'est pas un gain uniforme. C'est un déplacement du coût, et il faut savoir vers quoi on le déplace.

Écrire dans un chunk déjà basculé

C'est possible

- Insérer, corriger ou supprimer dans une période passée au columnstore fonctionne. Ce n'est pas une opération interdite.

C'est coûteux

- L'opération suppose de défaire puis refaire le travail de compression sur les lots concernés. Le coût est sans commune mesure avec la même écriture en rowstore.

Donc le délai de bascule est une décision

- Il doit dépasser la fenêtre pendant laquelle des données tardives arrivent encore — la valeur mesurée en M05, pour la troisième fois de la formation.

Une application qui corrige régulièrement des données de plus d'une semaine doit le savoir avant de choisir sept jours.



BLOC 9.2

Segmentation, tri, cardinalité

La décision qui divise le ratio par deux — ou fait échouer une correction —, et le lien direct avec la clé de série de M03.

segmentby : regrouper avant de compresser

Les lots ne mélangent pas les séries. Chaque lot ne contient que des lignes partageant la même valeur de segment.

Ce que cela apporte au volume

Une colonne dont toutes les valeurs sont identiques dans un lot ne coûte presque rien. Les colonnes voisines deviennent plus homogènes, donc compressent mieux.

Ce que cela apporte aux requêtes

Un filtre sur la colonne de segment devient une élimination de lots entiers, sans décompression. C'est le mécanisme le plus rentable du module.

Le lien avec M03

La colonne de segment est presque toujours la clé d'identification de série tranchée à la décision 2 de M03. Ce n'est pas une coïncidence : c'est la raison pour laquelle cette décision avait été présentée comme engageant six modules plus loin. Un identifiant textuel répété se paie ici, sur le ratio.

La cardinalité de segment

Trop fine

Beaucoup de valeurs distinctes, donc beaucoup de lots, chacun contenant peu de lignes. La compression n'a plus de matière sur laquelle travailler.

Le ratio s'effondre, et le volume de métadonnées explose.

Trop grossière

Peu de valeurs distinctes, donc des lots énormes mélangeant des séries hétérogènes.

Le ratio reste correct, mais lire un point impose de décompresser un lot immense.

Juste

Assez de lignes par lot pour que la compression travaille, assez de sélectivité pour que le filtre élimine des lots.

Le point optimal se mesure. Il dépend du volume, du nombre de séries et du profil de requêtes.

MISTRAL ne peut pas démontrer cet effet 490 séries, c'est une cardinalité triviale. L'atelier introduit un second jeu à vingt mille séries, uniquement pour cela.

orderby : l'ordre à l'intérieur du lot

```
timescaledb.compress_orderby = 'ts DESC'
```

Effet sur la compression

- Des valeurs voisines dans le temps se ressemblent. Les trier avant de compresser améliore nettement le ratio sur les colonnes de mesure.

Effet sur la dernière valeur

- Avec un tri décroissant sur le temps, la valeur la plus récente est en tête de lot. La requête de M06 en tire directement parti.

Effet sur les métadonnées de lot

- Les bornes minimum et maximum des colonnes de tri deviennent très sélectives, ce qui rend l'élimination de lots efficace.

Le piège

- Trier sur une colonne rarement filtrée ne sert à rien et coûte à la bascule. L'ordre doit refléter le profil de lecture, pas une intuition.

Éliminer des lots sans les lire

Les métadonnées calculées à la bascule permettent de savoir, sans décompresser, qu'un lot ne peut pas contenir la valeur cherchée.

1

Le prédicat porte sur une colonne de segment ou de tri

C'est la condition d'entrée. Un filtre sur une colonne quelconque ne bénéficie pas du mécanisme.

2

Les bornes du lot sont comparées au prédicat

Le minimum et le maximum du lot suffisent à trancher pour l'ensemble des lignes qu'il contient.

3

Le lot est écarté, ou décompressé

Un lot écarté n'est jamais lu. Le gain se lit dans le plan, sur le nombre de lots traités.

C'est la même logique que l'exclusion de chunks de M04, un cran plus bas : là c'était le chunk, ici c'est le lot.

Comment trancher, concrètement

1

Lister les prédicats réellement utilisés

Prendre les requêtes de M06 et de la pyramide de M08, pas des requêtes imaginaires.

2

Candidater la colonne la plus filtrée comme segment

Généralement l'identifiant de série. En vérifier la cardinalité avant de conclure.

3

Mesurer trois configurations au minimum

Ratio, durée des requêtes de référence, coût d'une correction. Trois grandeurs, pas une.

4

Vérifier sur un jeu de cardinalité différente

Le bon choix sur 490 séries n'est pas forcément le bon sur 20 000. C'est l'objet du second jeu.

5

Écrire la décision et le chiffre qui la porte

La configuration retenue doit vivre dans le dépôt, à côté du schéma, avec sa mesure.

Aucune de ces cinq étapes n'est une règle empirique. Ce sont cinq mesures, et une décision écrite.



ATELIER L08

Segmentation comparée

Trois configurations sur MISTRAL, puis la même sur vingt mille séries. C'est là que la cardinalité se démontre.

Brief

SOCLE · pour tous

- Trois configurations sur mesures : segment sur series_id, sur machine_id, aucun segment
- Mesurer pour chacune : ratio, durée des trois requêtes de référence, coût d'une correction
- Rejouer la configuration series_id sur mesures_hc et comparer le ratio obtenu
- Expliquer l'écart par la cardinalité de segment
- Trancher la configuration MISTRAL et l'inscrire avec sa mesure

EXTENSION · pour aller plus loin

- Relever le nombre de lots écartés avant et après ajout d'un prédicat de segment
- Mesurer le coût d'un retour en rowstore sur un chunk de trente jours
- Comparer le ratio colonne par colonne, et expliquer pourquoi l'horodatage compresse le mieux
- Chiffrer l'économie annuelle à l'échelle de production, tarifs à l'appui

Réussi si l'écart de ratio entre la meilleure et la pire configuration est mesuré, et expliqué par la cardinalité de segment.

Le tableau à remplir

Grandeur relevée	series_id	machine_id	aucun	hc
Ratio de compression				
Volume après bascule				
R1 • point le plus récent				
R2 • fenêtre de 3 jours				
R3 • agrégation complète				
Coût d'un UPDATE correctif				

La colonne hc n'est renseignée que sur la première ligne. C'est elle qui porte toute la démonstration de cardinalité.

Pièges et indices

La bascule échoue ou traîne sur les chunks récents

- Un chunk encore alimenté coûte cher à basculer. Ne compresser que des chunks au-delà de la fenêtre de données tardives de M05.

Le ratio mesuré est bien meilleur qu'attendu

- Vérifier que le chunk mesuré est plein. Un chunk partiel donne un ratio flatteur et non représentatif.

Les trois configurations ne portent pas sur les mêmes chunks

- Comparer chunk par chunk, ou sur la même plage. Le ratio varie d'un chunk à l'autre selon la densité des données.

mesures_hc semble donner un mauvais ratio

- C'est le résultat attendu : $\times 5,6$ contre $\times 9,3$ sur MISTRAL. Le chiffre qui porte la leçon est le nombre de lignes par segment et par jour — 96 contre 8 640 : vingt mille séries produisent des lots trop petits pour que la compression travaille pleinement.

État attendu

Livrable du module

- Configuration de compression tranchée
- Écart de ratio mesuré et expliqué
- État de reprise mistral-M09

Carte des concepts

10 / 12

Hypercore instancié, et la cardinalité enfin démontrée, pas seulement nommée.

Resynchronisation

- La ligne mesures_hc du tableau est la seule qui compte réellement : sans elle, la cardinalité reste une affirmation
- Un participant qui n'a pas fini les trois configurations restaure mistral-M09 et garde ses mesures partielles
- Le délai de bascule retenu doit être cohérent avec le retard d'arrivée de M05 : le vérifier explicitement

CE QUI SUIV

M10 • Cycle de vie de la donnée

Le volume est divisé. Reste à décider ce qu'on garde, combien de temps, et sous quelle forme.

1

Rétention par suppression de chunk

Instantanée là où un DELETE massif est destructeur. La pathologie n° 2 de M01 se referme ici.



2

Downsampling avant purge

Garder l'agrégat, jeter le détail — et vivre avec l'irréversibilité de ce choix.



3

L'ordre des trois politiques

Rafraîchir après avoir purgé la source produit un agrégat vide, silencieusement.

Une fonction qui n'existe qu'en managé

Le stockage étagé vers le stockage objet est propre au service managé. En auto-hébergé, il n'existe pas — M10 le traite en démonstration, et le dit.



MODULE M10

Cycle de vie de la donnée

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M09

- Configuration de compression tranchée et mesurée
- L'effet de la cardinalité est chiffré, plus seulement nommé
- Le délai de bascule est cohérent avec le retard d'arrivée

Ce qui manque encore

- Rien n'a jamais été supprimé : la fenêtre grandit indéfiniment
- Trois politiques vont coexister, et leur ordre n'est pas indifférent
- Le coût du stockage n'a jamais été mis en face d'une granularité

Objectif du module articuler compression, rétention et rafraîchissement dans le bon ordre, et chiffrer l'arbitrage coût contre granularité.

Agenda et objectifs

1 Rétention et downsampling

2 Articulation des politiques et coût

L Atelier L09 · la chaîne complète

✓ Point d'acquis

La pathologie n° 2 de M01 se referme ici

« La purge des données anciennes bloque la production une nuit par mois. » Le module y répond en une ligne de politique — et la carte des concepts se complète à douze sur douze.



BLOC 10.1

Rétention et downsampling

Supprimer sans bloquer, et décider ce qu'on garde du détail avant de le jeter.

Supprimer un chunk, ou supprimer des lignes

DELETE sur un intervalle

Chaque ligne est marquée morte, le journal enregistre l'opération, l'espace n'est pas rendu. L'autovacuum doit ensuite passer sur toute la table.

Durée proportionnelle au volume, verrous longs, bloat garanti.

Suppression de chunk

Le chunk est une table : on la supprime. L'espace est rendu immédiatement, sans parcours ni journal de ligne.

Durée quasi constante, quel que soit le volume contenu.

```
SELECT add_retention_policy('mesures', INTERVAL '90 days');  
-- et pour un besoin ponctuel, hors politique :  
SELECT drop_chunks('mesures', older_than => INTERVAL '90 days');
```

Ce que la suppression de chunk implique

La granularité est celle du chunk

- On ne supprime pas « les données de plus de 90 jours », on supprime les chunks entièrement antérieurs. Avec des chunks de sept jours, jusqu'à six jours de données survivent au-delà de la limite.

C'est irréversible et immédiat

- Aucune corbeille, aucun retour arrière. Un appel manuel avec un mauvais intervalle détruit des données en quelques secondes.

Les agrégats ne suivent pas automatiquement

- Supprimer la source ne supprime pas ce qui en a été matérialisé — et ne le protège pas non plus. C'est tout l'objet du bloc suivant.

La granularité de rétention est une conséquence directe de l'intervalle de chunk tranché en M04. Une décision en appelle une autre.

BLOC 10.1

Downsampling : garder l'agrégat, jeter le détail

La pyramide de M08 rend la purge possible : le détail peut disparaître parce que l'essentiel a déjà été matérialisé plus haut.

Donnée	Conservée	Rétention retenue
Mesures brutes, rowstore	0,1 Hz, tout le détail	7 jours
Mesures brutes, columnstore	0,1 Hz, tout le détail	90 jours
Agrégat minute	moyenne, somme, compte, max	1 an
Agrégat horaire	idem	10 ans
Agrégat journalier	idem	sans limite

Le downsampling est irréversible : la décision doit être datée, documentée, et validée par le métier, pas prise par l'exploitation.



BLOC 10.2

Articulation des politiques et coût

Trois politiques sur la même hypertable, et un ordre qui n'est pas indifférent.

L'ordre des trois politiques

Rafraîchir un agrégat après avoir purgé sa source produit un agrégat vide. Silencieusement.

1

Rafraîchir l'agrégat

Il doit avoir matérialisé une période avant que sa source ne disparaisse.

2

Basculer en columnstore

Sur des chunks au-delà de la fenêtre de données tardives.

3

Purger la source

En dernier, et seulement au-delà de la fenêtre couverte par l'agrégat.

La contrainte n'est pas dans l'ordonnanceur : elle est dans le choix des fenêtres. Une rétention plus courte que la portée du rafraîchissement suffit à produire le vide.

L'agrégat est une hypertable comme une autre

Il grossit, il vieillit, et il mérite ses propres politiques → ce que l'on oublie systématiquement.

```
-- compression de l'agregat lui-meme
ALTER MATERIALIZED VIEW mistral_1min SET (
  timescaledb.compress = true);
SELECT add_compression_policy('mistral_1min', INTERVAL '30 days');

-- retention de l'agregat lui-meme
SELECT add_retention_policy('mistral_1min', INTERVAL '1 year');
```

Pourquoi c'est fréquemment oublié

Sur MISTRAL, le niveau minute non compressé (4,1 Go) pèse plus que les 45 jours de brut en columnstore (4,0 Go). Sans politique propre, l'agrégat devient le premier poste de stockage, après avoir été créé pour en économiser.

Le stockage étagé n'existe pas partout

Le stockage étagé vers le stockage objet est une architecture propre au service managé. En auto-hébergé, la fonction n'existe pas dans l'extension.

En service managé

Les chunks anciens partent sur un stockage objet bien moins cher, et restent interrogeables par la même requête, plus lentement.

Démonstration en séance sur l'instance managée.

En auto-hébergé

Il faut construire l'équivalent : exporter, supprimer, et fournir un chemin de relecture. C'est un projet, pas un réglage.

Ou accepter que la rétention soit la seule réponse.

Le dire avant que quelqu'un ne construise une architecture de rétention sur une fonction dont il ne dispose pas.

Coût contre granularité, à l'échelle de production

Scénario de rétention	Volume à 3 ans	Ce qu'on perd
Tout le détail, rowstore	très élevé	rien
Tout le détail, columnstore	élevé	rien
Détail 90 jours + agrégats	modéré	le détail au-delà de 90 jours
Détail 7 jours + agrégats	faible	toute analyse fine rétrospective

La question à poser au métier « à quand remonte la dernière fois où vous avez eu besoin du détail à la seconde sur une donnée de plus de trois mois ? »



ATELIER L09

La chaîne complète

Trois politiques sur MISTRAL, dans le bon ordre, et l'économie chiffrée.

Brief

SOCLE · pour tous

- Appliquer la chaîne : 7 jours en rowstore, 90 jours en columnstore, agrégats horaires 10 ans — et compresser le niveau minute, qui pèse plus que le brut compressé
- Vérifier l'état des chunks après exécution des trois politiques
- Chiffrer l'économie obtenue sur les deux échelles
- Vérifier que la rétention ne coupe pas sous la portée du rafraîchissement

EXTENSION · pour aller plus loin

- Provoquer l'erreur d'ordre : purger la source avant rafraîchissement, et constater l'agrégat vide
- Appliquer la rétention à l'agrégat continu lui-même, et mesurer le gain
- Chiffrer l'économie annuelle avec et sans stockage étagé, tarifs à l'appui
- Mesurer la durée d'une suppression de chunk face à un DELETE équivalent

Réussi si les politiques s'exécutent, l'état des chunks est conforme à la cible, et l'économie est chiffrée sur les deux échelles.

Pièges et indices

La rétention ne supprime rien

- Seuls les chunks entièrement antérieurs à la limite partent. Et sur un jeu daté, une limite relative à `now()` ne coupe rien : passer par `drop_chunks` et `compress_chunk` à borne explicite, calculée depuis `max(ts)`.

L'agrégat devient vide

- La rétention coupe sous la portée du rafraîchissement. C'est l'erreur d'ordre du bloc 10.2, et elle ne produit aucun message.

Le stockage étagé n'est pas disponible

- Il ne l'est qu'en service managé. La démonstration se fait sur l'instance partagée, pas sur le conteneur local.

L'économie chiffrée paraît trop belle

- Vérifier l'échelle. Sur les deux jours de l'atelier, l'extrapolation à trois ans amplifie toute erreur de calcul par un facteur cinq cents.

État attendu

Livrable du module

- politiques.sql : la chaîne complète
- Économie chiffrée sur les deux échelles
- État de reprise mistral-M10

Carte des concepts

12 / 12

Politique et job instanciés. Les douze notions de M01 sont désormais toutes vécues.

Resynchronisation

- La carte est complète : c'est le moment de la relire en entier, notion par notion, avant d'aborder l'exploitation
- Trois des six pathologies de M01 sont résolues, et deux le seront en M12 et M15
- Un participant dont les politiques ne s'exécutent pas doit vérifier les workers avant tout le reste

M11 · Automatisation par les jobs

Cinq politiques tournent désormais sur MISTRAL. Personne ne sait ce qui se passe si l'une d'elles échoue.

1

Les politiques sont des jobs

Compression, rétention, rafraîchissement partagent le même ordonnanceur, la même file, le même historique.

2

Écrire ses propres actions

Contrôle qualité, purge sélective, alerte sur capteur muet — le même mécanisme, pour du métier.

3

Un job qui échoue est silencieux

C'est le mode de défaillance le plus courant, et il ouvre le catalogue de pannes de M15.

Changement de perspective

Les dix premiers modules concevaient. Les cinq suivants exploitent — et la première question de l'exploitation est : comment sait-on que ça marche encore ?



MODULE M11

Automatisation par les jobs

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M10

- La chaîne complète tourne : rafraîchissement, bascule, rétention
- L'économie est chiffrée sur les deux échelles
- Les douze notions de M01 sont instanciées

Ce qui manque encore

- Cinq travaux automatiques tournent, et personne ne les a jamais regardés
- Rien n'avertit si l'un d'eux cesse de s'exécuter
- Aucune tâche métier n'est automatisée : tout se fait à la main

Changement de perspective les dix premiers modules concevaient. Les cinq suivants exploitent, et la première question est : comment sait-on que ça marche encore ?

Agenda et objectifs

1 L'ordonnanceur et les jobs existants

2 Écrire ses propres actions

L Atelier L10 · action de contrôle qualité

✓ Point d'acquis

Le mode de défaillance central de l'exploitation

Un job qui échoue ne produit ni erreur applicative, ni alerte, ni ralentissement visible. Il produit un silence, et une donnée qui cesse d'être à jour sans que personne ne s'en aperçoive.



BLOC 11.1

L'ordonnanceur et les jobs existants

Cinq travaux tournent déjà sur MISTRAL. Les regarder avant d'en ajouter.

Cinq travaux, un seul mécanisme

Tout ce qui a été mis en place depuis M08 s'exécute par le même ordonnanceur, dans la même file, avec le même historique.

```
SELECT job_id, proc_name, schedule_interval, hypertable_name
FROM   timescaledb_information.jobs
ORDER BY job_id;
```

Rafraîchissement minute

mis en place en M08

Rafraîchissement heure

mis en place en M08

Rafraîchissement jour

mis en place en M08

Bascule columnstore

mis en place en M09

Rétention

mis en place en M10

Une seule chose à surveiller plutôt que cinq crons dispersés. C'est le principal argument du mécanisme — et il a une limite, au dernier slide du bloc suivant.

Workers, file d'attente, concurrence

Un nombre fini de workers

- Le paramètre de M02 fixe combien de travaux peuvent s'exécuter simultanément. Au-delà, les autres attendent.

Un job long bloque une place

- Une matérialisation qui dure vingt minutes occupe un worker vingt minutes. Les rafraîchissements courts s'accumulent derrière.

Le plafond global l'emporte

- Si le nombre total de processus de travail ne couvre pas les workers de l'extension et le parallélisme, rien ne démarre — et rien ne le signale.

Le troisième point est la première panne du catalogue de M15, et il commence à la mise en service de M02.

L'historique d'exécution

```
SELECT job_id, last_run_started_at, last_successful_finish,  
       last_run_status, total_runs, total_failures, next_start  
FROM   timescaledb_information.job_stats  
ORDER  BY total_failures DESC, job_id;
```

```
-- le detail des erreurs, quand il y en a  
SELECT job_id, start_time, err_message  
FROM   timescaledb_information.job_errors  
ORDER  BY start_time DESC LIMIT 20;
```

Les deux colonnes qui comptent

`last_successful_finish`, parce qu'un job peut s'exécuter tous les quarts d'heure en échouant à chaque fois. Et `next_start`, parce qu'un job suspendu ne remonte aucune erreur → il ne se déclenche simplement plus.

Modifier, suspendre, forcer, supprimer

```
-- changer la frequence sans recreer la politique
SELECT alter_job(1005, schedule_interval => INTERVAL '15 minutes');

-- suspendre : le job reste defini, il ne se declenche plus
SELECT alter_job(1005, scheduled => false);

-- forcer maintenant, hors transaction : l'erreur revient au client, pas dans job_errors
CALL run_job(1005);

-- supprimer definitivement
SELECT delete_job(1005);
```

Le piège de la suspension un job suspendu pour une maintenance et jamais réactivé est indétectable par une supervision qui ne surveille que les erreurs. Il faut surveiller next_start, pas seulement last_run_status.



BLOC 11.2

Écrire ses propres actions

Le même mécanisme, pour des tâches métier.

Une action est une procédure à deux arguments

```
CREATE PROCEDURE controle_qualite(job_id int, config jsonb)
LANGUAGE plpgsql AS $$
DECLARE seuil interval := (config->>'seuil')::interval;
BEGIN
    INSERT INTO alertes_qualite (detecte_le, series_id, dernier_point)
    SELECT now(), series_id, max(ts)
    FROM mesures GROUP BY series_id
    HAVING max(ts) < (SELECT max(ts) FROM mesures) - seuil;
END $$;
```

La signature est imposée

Un identifiant de job, une configuration en JSON. C'est ce contrat qui permet à l'ordonnanceur de l'appeler.

C'est une procédure, pas une fonction

Elle peut donc valider ses propres transactions → utile pour un traitement long qu'on veut découper.

Enregistrer, paramétrer, planifier

```
SELECT add_job('controle_qualite', INTERVAL '1 hour',  
              config => '{"seuil": "1 hour"}'::jsonb);
```

```
-- verifier l'enregistrement et la prochaine execution  
SELECT job_id, proc_name, config, next_start  
FROM   timescaledb_information.jobs  
WHERE  proc_name = 'controle_qualite';
```

● La configuration vit dans la base

Changer le seuil ne suppose ni redéploiement ni modification de code : un `alter_job` suffit.

● Le job s'exécute sous le rôle qui l'a créé

Ce n'est pas neutre : il peut voir des données qu'un lecteur ordinaire ne voit pas. Le sujet revient en M14.

Quatre cas MISTRAL

1

Contrôle qualité

Détecter les capteurs sans donnée depuis plus d'une heure par rapport au dernier point du parc, et consigner le constat.

L'atelier

2

Purge sélective

Supprimer les mesures d'un site déclassé, hors du cadre de la politique globale.

M10

3

Recalcul conditionnel

Forcer le rattrapage d'un agrégat après un backfill détecté.

M08

4

Contrôle de cohérence

Comparer périodiquement un agrégat à son calcul direct sur un échantillon.

M15

Le quatrième est le plus sous-estimé : c'est lui qui aurait détecté l'agrégat vide du bloc 10.2.

Dans la base, ou dans cron ?

Dans la base

- L'automatisation suit la sauvegarde et la restauration
- Un seul historique d'exécution, requêtable en SQL
- Pas de machine tierce à maintenir

La limite du raisonnement

- Un job ne survit pas à une instance indisponible : il ne rattrape pas les exécutions manquées
- Rien n'orchestre entre plusieurs bases, ni entre la base et le reste du système
- Le versionnement du code d'une action reste à la charge de l'équipe

La règle applicable ce qui concerne le cycle de vie de la donnée va dans la base. Ce qui orchestre plusieurs systèmes reste dehors, dans un ordonnanceur qui sait rattraper.



ATELIER L10

Action de contrôle qualité

Écrire, planifier, superviser, et provoquer une erreur pour la retrouver.

Brief

SOCLE · pour tous

- Écrire une action détectant les capteurs sans donnée depuis plus d'une heure
- L'enregistrer avec sa configuration, et la planifier
- Vérifier son exécution et sa prochaine échéance dans le catalogue
- Provoquer une erreur volontaire en arrière-plan et la retrouver dans le journal des erreurs

EXTENSION · pour aller plus loin

- Écrire une purge sélective paramétrée par site, et la piloter par sa configuration
- Mesurer l'effet d'un job long sur l'exécution des politiques concurrentes
- Instrumenter l'action pour qu'elle expose sa durée et son résultat en table
- Écrire le contrôle de cohérence qui aurait détecté l'agrégat vide de M10

Réussi si le job s'exécute selon sa planification, son historique est consultable, et l'erreur volontaire apparaît bien dans le journal.

Pièges et indices

Le job ne se déclenche jamais

Vérifier le nombre de workers avant tout le reste. C'est la première cause, et elle ne produit aucun message.

L'action fonctionne à la main, pas en job

Le contexte d'exécution diffère : rôle, search_path, privilèges. Tester en appelant run_job, pas la procédure directement.

L'erreur volontaire n'apparaît nulle part

Elle est dans le journal des erreurs de l'extension, pas dans celui de PostgreSQL — et seul l'ordonnanceur y écrit. Un CALL run_job() en session renvoie l'erreur au client sans rien journaliser.

Le job s'exécute mais ne fait rien

Une configuration mal formée est lue sans erreur et donne une valeur nulle. Journaliser les paramètres reçus au début de l'action.

État attendu

Livrable du module

- Une action de contrôle qualité planifiée
- L'erreur volontaire retrouvée dans le journal
- État de reprise mistral-M11

Six jobs sur MISTRAL

6

Cinq politiques et une action métier. Toutes surveillables par une seule requête.

Resynchronisation

- La requête sur job_stats est le premier élément du jeu de diagnostic construit en M15 : la conserver
- Un job qui n'a jamais échoué mais dont next_start est nul est suspendu, pas sain
- L'action écrite ici sera reprise en M15 comme source d'une des quatre alertes

M12 • Migration

La cible est complète : schéma, dimensionnement, agrégats, compression, rétention, automatisation. Reste à y amener la production.

1

Cinq stratégies, un critère

Le budget d'interruption décide, le volume ne fait qu'ajuster. Dump, copie, réplication, double écriture, conversion.

2

La checklist avant l'exécution

Elle est le livrable de l'atelier.
L'exécution n'en est que la vérification.

3

La procédure de repli

La question qu'on oublie de préparer, et celle qui coûte le plus cher quand elle manque.

Une contrainte posée en M03 revient

mistral_legacy utilise le partitionnement déclaratif. Une table déjà partitionnée ne se convertit pas : il faudra recréer et recopier, pas convertir sur place.



MODULE M12

Migration

Amener la production sur la cible, avec un budget d'interruption maîtrisé

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M11

- Six travaux automatiques, surveillables par une requête
- Une action métier écrite, planifiée, et dont l'erreur a été retrouvée
- La cible est complète, de bout en bout

Ce qui manque encore

- La production tourne toujours sur `mistral_legacy`
- Deux cents gigaoctets à déplacer, et un budget d'interruption à négocier
- Aucune procédure écrite, donc aucune migration possible

Objectif du module produire une checklist de bascule éprouvée. L'exécution n'en est que la vérification.

Agenda et objectifs

- 1 Cinq stratégies, un critère
- 2 Conduire une migration à faible interruption
- L Atelier L11 · migration MISTRAL
- ✓ Point d'acquis

Le livrable n'est pas la migration, c'est la procédure

L'atelier impose d'écrire la checklist complète (repli compris) avant de toucher quoi que ce soit. Une migration improvisée réussit parfois ; elle ne se refait pas.



BLOC 12.1

Cinq stratégies, un critère

Le budget d'interruption décide. Le volume ne fait qu'ajuster.

Cinq stratégies

Stratégie	Interruption	Complexité	Quand la retenir
Dump et restauration	heures	faible	Volume modeste, fenêtre de nuit acceptée
Copie parallélisée	dizaines de min	moyenne	Gros volume, fenêtre courte négociable — le chemin de l'atelier (plages + delta)
Réplication logique	minutes	élevée	Fenêtre de quelques minutes ; inutilisable vers une hypertable en 2.29
Double écriture applicative	nulle	très élevée	Aucune interruption tolérée, et du temps devant
Conversion sur place	variable	moyenne	Source et cible sur la même instance

La colonne complexité n'est pas un détail : une réplication logique mal maîtrisée coûte plus cher qu'une nuit d'interruption assumée. Et vers une hypertable, elle ne fonctionne pas en 2.29 : la copie initiale écrit dans la table racine, sans routage ni message — 3 Go copiés, 0 ligne visible.

Le budget d'interruption décide

La question n'est pas « combien de données », c'est « combien de temps le service peut-il s'arrêter ».

- **Le volume ajuste, il ne décide pas**

Il détermine la durée de la copie initiale, donc le coût de la préparation — pas la fenêtre de bascule.

- **La question se pose au métier, pas à l'exploitation**

Et elle se pose avant de choisir la stratégie, pas après avoir commencé à copier.

- **Une réponse honnête est souvent plus large qu'on ne croit**

« Deux heures un dimanche matin » débloque une stratégie bien moins risquée qu'une réplication logique.

La contrainte posée en M03

mistral_legacy utilise le partitionnement déclaratif. Une table déjà partitionnée ne se convertit pas en hypertable.

Ce que cela élimine

La conversion sur place. Il n'y a pas de chemin qui transforme la table existante là où elle est.

Ce que cela impose

Créer la cible à côté, y recopier les données, puis basculer. C'est une migration au sens plein, pas une modification.

La bonne nouvelle

Recréer plutôt que convertir permet d'appliquer d'emblée toutes les décisions des dix modules précédents : schéma de M03, intervalle de M04, index de M06, segmentation de M09. Une conversion sur place aurait figé l'existant.



BLOC 12.2

Conduire une migration à faible interruption

Une séquence, trois contrôles, un point de bascule, et un repli écrit à l'avance.

La séquence type

1

Préparer la cible

Schéma complet, index, mais politiques désactivées et compression non activée. Une base dédiée : celle du fil rouge porte déjà un jeu qui n'est pas celui de la legacy.

2

Copie initiale

Le gros du volume, copié par plages depuis la source, sans interruption de service. Durée longue, sans conséquence.

3

Rattrapage continu

Un delta sur identifiant monotone rejoue le flux courant par plages, production ouverte. La cible converge vers la source.

4

Vérifier

Les trois contrôles de cohérence, avant toute décision de bascule.

5

Basculer

La seule étape qui interrompt. Elle doit être courte parce que tout le reste a été fait avant.

6

Activer et durcir

Compression, politiques, jobs. Après la bascule, jamais pendant.

Une seule des six étapes interrompt le service. Toute la difficulté consiste à déplacer du travail des étapes 5 vers les étapes 1 à 4.

Copie initiale : les leviers connus

Tout ce qui a été mesuré en M05 s'applique ici, à l'échelle de la production.

● Créer les index après la copie

Le levier le plus rentable, mesuré en M05. Il divise la durée d'un chargement initial.

● Ne pas compresser pendant

Activer la compression avant la copie multiplie la durée. Toujours après, à l'étape 6.

● Désactiver les politiques

Un rafraîchissement d'agrégat qui démarre au milieu de la copie travaille sur des données partielles.

● Paralléliser la copie

Un outil dédié découpe par plage temporelle. Le gain est réel, et le pilotage plus délicat.

Le détail qu'on oublie une fois, et une seule

```
-- AVANT pg_restore : suspend les mecanismes internes
SELECT timescaledb_pre_restore();

-- ... pg_restore ...

-- APRES : remet l'extension en etat de fonctionner
SELECT timescaledb_post_restore();
```

Sans l'appel préalable

La restauration échoue, ou produit un état incohérent : les mécanismes internes de l'extension interfèrent avec l'insertion des données restaurées.

Sans l'appel final

L'instance reste dans un mode dégradé. Les symptômes sont diffus et le diagnostic long, l'oubli le plus coûteux de la migration.

Trois contrôles, dans cet ordre

1

Comptage global

Rapide, et suffisant pour détecter une copie tronquée. Insuffisant pour tout le reste.

2

Somme de contrôle par période

Une agrégation par jour, comparée des deux côtés. Détecte les trous et les décalages.

3

Échantillonnage de lignes

Cent lignes tirées au hasard, comparées champ par champ. Détecte les altérations de valeur.

Choisir un contrôle stable une somme sur des flottants diverge entre deux instances si l'ordre d'agrégation diffère. Utiliser des comptages et des sommes sur entiers.

Le point de bascule

Les critères d'entrée

- Les trois contrôles passent
- Le retard de réplication est stable et faible
- La procédure de repli est écrite et relue

Ce qui doit être décidé avant

- Qui prononce la bascule, et en combien de temps
- Quel délai maximal avant de déclencher le repli
- Qui prévient les utilisateurs, et à quel moment

La décision est humaine, la vérification est automatique

Les critères d'entrée s'exécutent en une requête. La bascule elle-même se prononce, et le nom de celui qui la prononce figure dans la checklist.

La procédure de repli

C'est la partie qu'on n'écrit pas, et c'est celle qui coûte le plus cher quand elle manque.

- **Elle s'écrit avant la bascule**

Une procédure de repli rédigée pendant l'incident est une improvisation avec un nom rassurant.

- **Elle a un délai de déclenchement**

« Si la cible n'est pas opérationnelle en N minutes, on revient. » Le N est décidé à froid.

- **Elle suppose que la source reste intacte**

Ne rien supprimer côté source avant plusieurs jours d'exploitation nominale de la cible.

- **Elle doit traiter les écritures faites sur la cible**

Ce qui a été écrit après bascule et avant repli doit être récupéré, ou explicitement abandonné.

Ce qui ne se réplique pas, ou mal

Les séquences

- Ni par la réplication logique, ni par une copie par plages : repositionner la séquence (setval) au-delà du maximum observé. Découvert sinon après bascule, sur la première insertion.

Les changements de schéma

- Non répliqués non plus. Toute modification pendant la migration doit être appliquée des deux côtés.

Les index

- À recréer côté cible, et de préférence après la copie.

Les extensions manquantes

- La cible doit disposer de toutes les extensions utilisées par le schéma source.

Les privilèges et les rôles

- Ils appartiennent à l'instance, pas à la base. À rejouer explicitement.

Les deux premiers sont responsables de la majorité des incidents post-bascule. Ils figurent en toutes lettres dans la checklist de l'atelier.



ATELIER L11

Migration MISTRAL

La checklist d'abord, l'exécution ensuite. Et un repli écrit avant le démarrage.

Brief

SOCLE • pour tous

- Produire la checklist de bascule AVANT de toucher quoi que ce soit
- Préparer le schéma cible et lancer la copie initiale
- Rattraper par delta sur identifiant monotone, le flux d'écriture restant ouvert sur la source
- Exécuter les trois contrôles de cohérence
- Basculer, mesurer l'interruption réelle, puis activer compression et politiques

EXTENSION • pour aller plus loin

- Provoquer un échec pendant le rattrapage et exécuter le repli
- Migrer avec une extension manquante côté cible et diagnostiquer l'erreur
- Tenter la réplication logique depuis la source partitionnée (`publish_via_partition_root` obligatoire) et constater pourquoi elle échoue vers une hypertable
- Transposer la checklist à l'échelle de production : quels contrôles ne passent plus ?

Réussi si les trois contrôles passent, l'interruption est mesurée, et la checklist comportait une procédure de repli écrite avant le démarrage.

Ce que la checklist doit contenir

Avant

- Les prérequis vérifiés côté cible
- La procédure de repli, complète
- Le délai de déclenchement du repli
- Qui prononce la bascule

Pendant

- Chaque étape, avec son critère de passage
- Les trois contrôles et leurs seuils
- Le relevé de l'interruption réelle

Après

- L'activation de la compression et des politiques
- Les vérifications post-basculé
- La date à laquelle la source peut être supprimée

Elle est réutilisable telle quelle sur le contexte du participant. C'est le seul livrable de la formation conçu pour sortir de MISTRAL.

Pièges et indices

La compression a été activée avant la copie



La durée est multipliée. Toujours après la bascule, jamais pendant → c'est l'étape 6 de la séquence.

Les séquences sont restées à zéro côté cible



Elles ne sont pas répliquées. La première insertion post-bascule entre en conflit, et l'incident est immédiat.

La somme de contrôle diverge sans raison



Elle porte sur des flottants et l'ordre d'agrégation diffère. Recontrôler avec un comptage et une somme sur entiers.

La restauration laisse l'instance en mode dégradé



L'appel de fin d'opération a été oublié. Symptômes diffus, diagnostic long → le vérifier systématiquement.

État attendu

Livrable du module

- checklist-bascule.md, éprouvée
- Interruption réelle mesurée et consignée
- Les trois contrôles passés

Pathologies de M01

5 / 6

Le silo entre référentiel et mesures se referme ici : les deux vivent enfin dans la même base.

Resynchronisation

- La checklist est le seul livrable conçu pour sortir de MISTRAL : la relire avec son propre contexte en tête
- Un participant dont la bascule a échoué a produit le résultat le plus instructif de l'atelier — le faire raconter
- L'interruption mesurée en salle n'est pas transposable ; la procédure, elle, l'est

M13 · Sauvegarde, restauration, montée de version

La production tourne sur la cible. La première question est : que se passe-t-il si elle disparaît ?

1

Ce qu'une sauvegarde restaure vraiment

Un dump naïf peut rendre des tables ordinaires là où il y avait des hypertables.



2

La restauration à un instant donné

Principe, prérequis, durée réelle — mesurée, pas estimée.



3

La montée de version

Un chaînage imposé par la matrice de support, et aucun retour arrière possible.

Une règle qui vaut pour tout le module

Répéter la restauration est la seule preuve qu'une sauvegarde existe. Une sauvegarde jamais restaurée est une hypothèse, pas une garantie.



MODULE M13

Sauvegarde, restauration, montée de version

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M12

- La production tourne sur la cible
- Une checklist de bascule éprouvée, réutilisable hors MISTRAL
- Cinq des six pathologies de M01 sont résolues

Ce qui manque encore

- Aucune sauvegarde n'a jamais été prise, ni restaurée
- Personne ne sait combien de temps prendrait une restauration
- L'instance est en 2.29 aujourd'hui ; elle ne le restera pas

La règle du module répéter la restauration est la seule preuve qu'une sauvegarde existe. Une sauvegarde jamais restaurée est une hypothèse.

Agenda et objectifs

1 Sauvegarde et restauration

2 Montée de version

L Atelier L12 · restauration à un instant donné

✓ Point d'acquis

Deux sujets qui n'ont rien en commun, sauf l'essentiel

Une restauration mal préparée et une montée de version mal ordonnée ont le même point commun : on ne s'en aperçoit qu'au moment où il est trop tard pour revenir en arrière.



BLOC 13.1

Sauvegarde et restauration

Ce qu'une sauvegarde restaure vraiment, et en combien de temps.

La sauvegarde restaurée fonctionne, mais...

Les hypertables sont redevenues des tables ordinaires. La base répond, les requêtes passent, et tout est lent.

Ce que l'extension ajoute vit dans son catalogue

- Chunks, dimensions, politiques, agrégats : ce sont des objets internes que l'outil de sauvegarde ne connaît pas comme tels.

Les chunks compressés compliquent encore

- Leur format interne n'est pas celui d'une table ordinaire. Une restauration naïve peut échouer, ou produire un état incohérent.

D'où une procédure spécifique

- Encadrer la restauration par les deux appels dédiés de l'extension. C'est le même mécanisme qu'en M12, pour la même raison.

Sauvegarde logique : la procédure correcte

```
# sur la source
pg_dump -Fc -f mistral.dump mistral

-- sur la cible, AVANT la restauration
SELECT timescaledb_pre_restore();

# puis
pg_restore -d mistral --no-owner mistral.dump

-- et APRES, sans exception
SELECT timescaledb_post_restore();
```

À quoi cette voie sert vraiment migrer, cloner un environnement, extraire une base. Pas à sauvegarder une production volumineuse : la durée de restauration croît avec le volume, sans plafond.

Sauvegarde physique et journaux

Sauvegarde de base

Une copie des fichiers de l'instance, prise à chaud. Sa durée dépend du volume, mais elle ne se rejoue pas ligne à ligne.

C'est la voie des productions volumineuses.

Archivage des journaux

Chaque enregistrement de journal est conservé. Rejoués sur la sauvegarde de base, ils reconstituent l'état à n'importe quel instant.

C'est ce qui rend possible la restauration à un instant donné.

Les deux paramètres à décider, et à écrire

Combien de perte de données est acceptable, ce qui fixe la fréquence d'archivage. Et combien de temps d'indisponibilité est acceptable, ce qui fixe la fréquence des sauvegardes de base. Ce sont deux questions métier, pas deux réglages.

Restaurer à un instant donné

1

Restaurer la sauvegarde de base

La plus récente antérieure à l'instant visé.

2

Fournir les journaux archivés

Ils seront rejoués depuis la fin de la sauvegarde de base.

3

Désigner l'instant cible

Un horodatage, ou un point de reprise nommé posé à l'avance.

4

Démarrer et laisser rejouer

La durée dépend du volume de journaux à rejouer, pas de la taille de la base.

5

Vérifier avant d'ouvrir

Les hypertables, leurs politiques et leurs agrégats — pas seulement le comptage des lignes.

L'étape 5 est celle qu'on saute. Une base restaurée dont les politiques ne repartent pas se dégrade silencieusement pendant des jours.

Réplicas, et ce qu'ils changent aux jobs

Réplica de haute disponibilité

Une copie prête à prendre le relais. Les travaux d'arrière-plan n'y tournent pas tant qu'il est en secours.

Après bascule, vérifier qu'ils ont bien démarré.

Réplica de lecture

Décharge les requêtes analytiques. Il n'exécute aucune politique et ne matérialise rien : tout vient du primaire.

Une lecture sur réplica peut donc voir un agrégat en retard.

Le cas du service managé les sauvegardes sont automatiques et le clonage à un instant donné se fait en quelques clics. Ce qui reste à la charge de l'exploitant : décider la rétention des sauvegardes, et vérifier régulièrement qu'un clone démarre réellement.



BLOC 13.2

Montée de version

Un chaînage imposé, un ordre à respecter, et aucun retour arrière.

La matrice de support commande le chemin

Version d'extension	PostgreSQL supportés	Conséquence
TimescaleDB 2.29	16, 17, 18	Version cible de la formation
TimescaleDB 2.28	15, 16, 17, 18	Dernière version couvrant PostgreSQL 15
Le chemin depuis une instance en PostgreSQL 15 ● Monter l'extension jusqu'à la dernière version qui supporte encore PostgreSQL 15 ● Puis monter PostgreSQL vers une version majeure supportée par la cible ● Puis seulement monter l'extension vers la version visée		

L'ordre des opérations

```
-- montee de l'extension seule
ALTER EXTENSION timescaledb UPDATE;

-- verifier ce qui est reellement charge
SELECT extversion FROM pg_extension WHERE extname = 'timescaledb';
```

L'extension doit être montée dans une session neuve

Une session ouverte avant la mise à jour continue d'utiliser l'ancienne bibliothèque chargée. Se reconnecter, systématiquement.

Le paquet et l'extension sont deux choses

Installer le paquet ne met pas la base à jour. Sans l'instruction de mise à jour, les deux versions divergent — c'est le piège de \dx vu en M02.

Le Toolkit se monte séparément

Il a son propre cycle de version. Une montée d'extension ne l'emporte pas.

Il n'y a pas de retour arrière

Une montée de version d'extension ne se redescend pas. La seule marche arrière est une restauration de sauvegarde.

Ce que cela impose avant

- Une sauvegarde fraîche, et surtout une sauvegarde dont on a déjà vérifié qu'elle se restaure. C'est la règle du bloc précédent, appliquée.

Ce que cela impose pendant

- Une instance de test montée d'abord, avec le même chemin de version et un jeu de données représentatif.

Ce que cela impose après

- Vérifier les politiques, les agrégats et les plans des requêtes critiques. Un plan qui change après montée de version est un incident silencieux.



ATELIER L12

Restauration à un instant donné

Provoquer une suppression accidentelle, revenir en arrière, et chiffrer la durée.

Brief

SOCLE • pour tous

- Prendre une sauvegarde de base et vérifier l'archivage des journaux
- Provoquer une suppression accidentelle, en notant l'instant précis
- Restaurer à l'instant précédant l'incident
- Vérifier que les hypertables, leurs politiques et leurs agrégats sont intacts
- Chiffrer la durée de restauration

EXTENSION • pour aller plus loin

- Restaurer un dump logique d'une base contenant des chunks compressés, et constater ce qui est perdu
- Monter l'extension d'une version mineure sur une instance de test et vérifier le catalogue
- Établir le chemin de montée complet d'une instance fictive en 2.21 sur PostgreSQL 15
- Comparer les plans des requêtes critiques avant et après montée de version

Réussi si la donnée est retrouvée, les politiques survivent et repartent, et la durée de restauration est chiffrée.

Pièges et indices

Les hypertables sont devenues des tables ordinaires

Les deux appels encadrant la restauration ont été omis. Recommencer : il n'y a pas de rattrapage après coup.

Les données sont là, mais les politiques ne repartent pas

Vérifier les six jobs, leur planification et leur prochaine échéance. C'est l'étape 5 de la séquence, et c'est celle qu'on saute.

La durée mesurée n'a rien à voir avec la production

Elle croît avec le volume de journaux à rejouer. La mesurer ici, puis extrapoler explicitement — et le dire dans le chiffre consigné.

L'instant cible tombe après l'incident

Vérifier l'horodatage relevé au moment de la suppression, et se souvenir que la restauration s'arrête juste avant l'instant demandé.

État attendu

Livrable du module

- Procédure de restauration éprouvée
- Durée de restauration chiffrée
- Chemin de montée de version documenté

La question de contrôle

« À quand remonte votre dernière restauration réellement exécutée, et combien de temps a-t-elle pris ? »

Resynchronisation

- La vérification des politiques après restauration est la seule étape que tout le monde saute : insister
- La durée mesurée en salle n'est pas transposable ; la procédure et la checklist de vérification le sont
- Le chemin de montée de version est à établir pour l'instance réelle du participant, pas pour MISTRAL

CE QUI SUIV

M14 · Sécurité et conformité

L'instance est sauvegardée et restaurable. Reste à décider qui peut voir quoi, et à répondre à une demande d'effacement.

1

Cloisonner, et en mesurer le coût

Trois rôles, un cloisonnement par site, et le surcoût sur les plans, chiffré, pas supposé.



2

Les privilèges et les chunks

Ce qui se propage aux chunks créés après coup, et ce qui ne se propage pas.



3

Le droit à l'effacement

Face à trois obstacles : la compression, les agrégats, les sauvegardes.

Un sujet que M11 a laissé ouvert

Un job s'exécute sous le rôle qui l'a créé. Sur une instance cloisonnée, cela contourne le cloisonnement, et il faut le traiter explicitement.



MODULE M14

Sécurité et conformité

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

Où en est le fil rouge

Acquis en M13

- Une restauration éprouvée, et sa durée chiffrée
- Une checklist de vérification post-restauration
- Le chemin de montée de version documenté

Ce qui manque encore

- Un seul rôle voit tout : aucun cloisonnement n'existe
- Les jobs de M11 s'exécutent sous le rôle qui les a créés
- Un client demande l'effacement de ses données. Personne ne sait répondre

Objectif du module cloisonner et en mesurer le coût, puis traiter le droit à l'effacement face à trois obstacles qui s'additionnent.

Agenda et objectifs

- 1 Cloisonner l'accès, et le mesurer
- 2 Chiffrement, conformité, effacement
- L Atelier L13 · cloisonnement MISTRAL
- ✓ Point d'acquis

Une particularité des données temporelles

Elles sont volumineuses, compressées, agrégées et sauvegardées. Chacune de ces qualités, acquise dans les modules précédents, rend l'effacement plus difficile. La conformité paie ce que la performance a gagné.



BLOC 14.1

Cloisonner l'accès, et le mesurer

Trois rôles, un cloisonnement par site, et un surcoût qui se chiffre.

Trois rôles, trois besoins distincts

Ingestion

Écrit dans les hypertables de mesures et d'événements. Ne lit rien, ne modifie aucun schéma.

Le compte le plus exposé : il vit dans les collecteurs.

Lecture

Interroge les agrégats et les mesures, éventuellement restreint à un périmètre.

Le plus nombreux : tableaux de bord, analystes, exports.

Administration

Modifie le schéma, gère les politiques et les jobs.

Le plus puissant, et celui dont l'usage doit être tracé.

La règle de séparation un collecteur compromis ne doit pas permettre de lire l'historique complet, ni de supprimer quoi que ce soit. C'est le seul argument qui compte pour séparer ingestion et lecture.

Privilèges et chunks

```
CREATE ROLE mistral_lecture NOLOGIN;  
GRANT USAGE ON SCHEMA public TO mistral_lecture;  
GRANT SELECT ON mesures, evenements, actifs TO mistral_lecture;  
  
-- verifier ce que voit reellement le role  
SET ROLE mistral_lecture;  
SELECT count(*) FROM mesures;  
RESET ROLE;
```

Le privilège porte sur l'hypertable

Il se propage aux chunks, y compris à ceux créés après l'attribution. C'est le comportement attendu, et il évite un piège classique du partitionnement manuel.

Les agrégats continus sont des objets distincts

Un droit de lecture sur les mesures ne donne pas accès à la pyramide. Les accorder explicitement, sinon les tableaux de bord tombent.

Cloisonner par site : la vue barrière

```
-- ENABLE ROW LEVEL SECURITY est refuse sur une hypertable compressee (2.29)
CREATE VIEW mesures_site WITH (security_barrier) AS
  SELECT m.* FROM mesures m WHERE m.series_id IN (
    SELECT af.series_id FROM affectation_capteur af
    JOIN actifs a ON a.machine_id = af.machine_id
    WHERE a.site_id = current_setting('mistral.site')::int);
GRANT SELECT ON mesures_site TO mistral_lecture; -- apres REVOKE sur mesures
```

Ce que cela garantit

Le rôle de lecture ne voit que la vue : le cloisonnement s'applique à chaque requête, quelle que soit l'application qui interroge.

Ce que cela coûte

Le prédicat s'ajoute à toutes les requêtes, et la vue casse certains plans : ×143 sur « dernière valeur par série », ×0,8 sur les fenêtres.

La RLS native est refusée en 2.29 sur une hypertable compressée et sur les matérialisations d'agrégats. Le cloisonnement applicatif, lui, ne protège de rien en session directe.

Mesurer le surcoût, plutôt que le supposer

1

Mesurer sans la sécurité au niveau ligne

Les trois requêtes de référence, protocole de M02.

2

Activer, puis remesurer

Sans rien changer d'autre. Le rapport est le surcoût.

3

Comparer les plans

Le prédicat de cloisonnement apparaît-il comme un filtre tardif, ou est-il poussé au plus près de la lecture ?

4

Vérifier l'élimination de lots

Sur les chunks en columnstore, comparer le nombre de lots traités avant et après.

Un cloisonnement dont le coût n'a pas été mesuré sera désactivé un jour, en urgence, par quelqu'un qui cherche de la performance.

Les jobs contournent le cloisonnement

Un job s'exécute sous le rôle qui l'a créé. Une action écrite par un administrateur voit tout, quelle que soit la politique de cloisonnement.

Ce n'est pas un défaut



Une politique de rétention doit pouvoir supprimer des chunks de tous les sites. Le comportement est nécessaire.

C'est un angle mort



Une action métier écrite à la hâte sous un compte d'administration devient un canal de lecture non cloisonné.

La réponse



Créer les actions métier sous un rôle dédié, doté du strict nécessaire, et le vérifier, plutôt que de le supposer.



BLOC 14.2

Chiffrement, conformité, effacement

Ce que la performance a gagné, la conformité le paie.

Chiffrement, secrets, réseau

En transit

Chiffrement de la connexion, imposé côté serveur et vérifié côté client.

Le minimum, y compris en réseau interne

Au repos

Chiffrement du volume, ou de l'instance en service managé. Transparent pour l'application.

Répond à une exigence, pas à une menace interne

Secrets

Les mots de passe des collecteurs et des jobs. Rotation planifiée, stockage hors du dépôt.

Le point faible réel de la plupart des installations

Réseau

Filtrage des adresses autorisées. Appairage de réseaux ou liaison privée en cloud.

Pertinent quand il remplace l'exposition publique, pas quand il s'y ajoute

Le droit à l'effacement, et trois obstacles

La compression

Effacer une ligne dans un chunk en columnstore suppose de défaire puis refaire les lots concernés. C'est possible, et coûteux.

Réponse : traiter par lots, hors heures de charge.

Les agrégats

La donnée a été absorbée dans des seaux matérialisés. La retirer suppose de rematérialiser les périodes concernées.

Réponse : rafraîchissement forcé après effacement.

Les sauvegardes

Les sauvegardes conservent l'état antérieur. Aucun effacement ne les atteint rétroactivement.

Réponse : la rétention des sauvegardes borne le délai.

Ce qu'on écrit au délégué à la protection des données l'effacement est effectif en base sous N jours, et purgé des sauvegardes au terme de leur rétention, soit M jours. Les deux délais sont documentés.

Pseudonymiser à l'ingestion plutôt qu'effacer après

Effacer après coup

Chercher, défaire la compression, rematérialiser les agrégats, attendre l'expiration des sauvegardes.

Coûteux, long, et jamais complet immédiatement.

Pseudonymiser à l'entrée

La donnée identifiante ne rentre jamais : elle est remplacée par un identifiant technique, et la table de correspondance vit ailleurs.

Effacer revient alors à supprimer une ligne de correspondance.

La décision se prend en M03, pas en M14

C'est un choix de modélisation. Le prendre au moment de la conformité suppose de reprendre le schéma, de recharger l'historique, et de refaire la migration. Sur MISTRAL, la question ne se pose pas — sur des données de personnes, elle est centrale.



ATELIER L13

Cloisonnement MISTRAL

Trois rôles, un cloisonnement par site, trois tentatives de contournement, et un surcoût mesuré.

Brief

SOCLE · pour tous

- Créer les trois rôles : ingestion, lecture, administration
- Mettre en place le cloisonnement par site sur les mesures et les trois niveaux d'agrégats, par vues barrière
- Valider qu'un exploitant d'un site ne voit pas les données d'un autre
- Résister aux trois tentatives de contournement fournies
- Mesurer le surcoût sur les plans, avant et après

EXTENSION · pour aller plus loin

- Mesurer l'interaction entre cloisonnement et columnstore : ratio et lots éliminés
- Effacer les données d'un actif dans un chunk compressé et dans les trois niveaux d'agrégats, et chiffrer
- Recréer l'action de M11 sous un rôle dédié et vérifier qu'elle ne voit plus tout
- Écrire la note qui explique à un délégué à la protection des données les délais d'effacement

Réussi si le cloisonnement résiste aux trois tentatives de contournement, et le surcoût est chiffré plutôt que supposé.

Pièges et indices

Le cloisonnement fonctionne sur les mesures, pas sur les agrégats

- Les agrégats continus sont des objets distincts, et leurs tables de matérialisation refusent la RLS. Sans vue barrière propre à chaque niveau, la pyramide est un canal de fuite complet.

Le propriétaire, et tout rôle qui garde un droit sur la table brute, passent à côté de la vue

- Vérifier les REVOKE sur mesures et sur chaque table de matérialisation. Tester avec un rôle dédié, jamais avec le compte d'administration.

Le job de M11 voit toujours tout

- C'est attendu : il s'exécute sous le rôle qui l'a créé. C'est l'objet de l'extension, et la troisième tentative de contournement.

Le surcoût mesuré est nul, ou négatif

- Sur les fenêtres, la sélectivité du site (180 séries sur 490) compense la vue. Sur la dernière valeur par série, le surcoût est massif : un tableau de bord cloisonné lit des agrégats cloisonnés.

État attendu

Livrable du module

- Trois rôles cloisonnés
- Surcoût mesuré sur les plans
- État de reprise mistral-M14

La question de contrôle

« Si votre collecteur était compromis ce soir, que pourrait lire l'attaquant, et que pourrait-il détruire ? »

Resynchronisation

- Le cloisonnement des agrégats est aussi important que celui des mesures : le vérifier avant la pause
- Un surcoût non mesuré sera invoqué un jour pour désactiver le cloisonnement en urgence
- La décision de pseudonymiser appartient à M03 : la signaler dans le plan d'application de ceux qui traitent des données de personnes

CE QUI SUIV

M15 · Diagnostic et supervision

Tout est en place. Un agrégat continu ne se rafraîchit plus depuis onze jours, et personne ne l'a vu.

1

Le jeu de requêtes de diagnostic

Construit depuis M02, morceau par morceau. Il se rassemble ici.

2

Sept pannes, une méthode

Symptôme, hypothèse, requête de vérification, correction, prévention.

3

Quatre alertes défendables

Celles qui se déclenchent sur situation provoquée, et jamais en régime normal.

Le dernier module, et la dernière pathologie

Deux pannes seront injectées à l'insu des participants. Le critère n'est pas de les résoudre, mais de les résoudre par une requête plutôt que par tâtonnement.



MODULE M15

Diagnostic et supervision

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

2 h 15 · ateliers L14 et L15 · prérequis : M08 et M11

Où en est le fil rouge

Acquis en M14

- Trois rôles cloisonnés, trois contournements refermés
- Le surcoût du cloisonnement est chiffré
- L'instance est conçue, migrée, sauvegardée, automatisée, cloisonnée

Ce qui manque encore

- Un agrégat ne se rafraîchit plus depuis onze jours, et personne ne l'a vu
- Aucune métrique n'est exposée, aucune alerte n'existe
- Les requêtes de diagnostic sont dispersées dans quatorze modules

Objectif du module diagnostiquer par une requête plutôt que par tâtonnement, et instrumenter de façon défendable dès le premier jour.

Agenda et objectifs

- 1 Le jeu de requêtes de diagnostic
- 2 Catalogue de pannes, méthode commune
- 3 Métriques, alertes, non-régression
- L Atelier L14 · pannes injectées
- L Atelier L15 · tableau de bord et alertes
- ✓ Point d'acquis

Le critère de L14 n'est pas de résoudre les deux pannes, mais de les résoudre par une requête, et de savoir laquelle.



BLOC 15.1

Le jeu de requêtes de diagnostic

Construit depuis M02, morceau par morceau. Il se rassemble ici.

Ce que vous avez déjà écrit

- | | |
|-----|---|
| M02 | Workers actifs, version chargée, configuration |
| M03 | Hypertables, dimensions, jointure au référentiel |
| M04 | Chunks et leurs bornes, temps de planification |
| M05 | Volume de journal, événements d'attente, retard d'arrivée |
| M08 | Contrôle de somme aux trois niveaux |
| M09 | Ratio de compression, lignes par segment |
| M11 | État des jobs, historique, journal des erreurs |

Le module ne les invente pas : il les rassemble dans un fichier unique, versionné, qui part avec le participant.

Les quatre familles de requêtes

```
-- 1. structure : ce qui existe
SELECT * FROM timescaledb_information.hypertables;
SELECT * FROM timescaledb_information.dimensions;

-- 2. volume : ce que ca pese, et comment c'est reparti
SELECT * FROM hypertable_detailed_size('mesures');
SELECT * FROM hypertable_compression_stats('mesures');

-- 3. automatisation : ce qui tourne, et ce qui echoue
SELECT * FROM timescaledb_information.job_stats;

-- 4. activite : ce qui se passe maintenant
SELECT wait_event_type, wait_event, count(*) FROM pg_stat_activity
WHERE state = 'active' GROUP BY 1,2;
```

Structure, volume, automatisation, activité. Toute panne du catalogue suivant se diagnostique par au moins une de ces quatre familles.



BLOC 15.2

Catalogue de pannes, méthode commune

Sept pannes, une seule méthode. Symptôme, hypothèse, requête, correction, prévention.

La méthode, en cinq temps

1

Symptôme

Ce que l'utilisateur voit, formulé sans hypothèse. « C'est lent » n'est pas un symptôme exploitable.

2

Hypothèses

Deux ou trois causes plausibles, classées par probabilité et par coût de vérification.

3

Requête de vérification

Celle qui départage. C'est le cœur de la méthode, et ce qui distingue le diagnostic du tâtonnement.

4

Correction

Et son effet mesuré, pas supposé.

5

Prévention

La requête qui aurait détecté l'incident trois jours plus tôt. Elle devient une alerte.

Le temps 5 est celui qu'on saute. C'est pourtant lui qui empêche l'incident de se reproduire, et lui qui alimente le bloc suivant.

Ce qui vient de la conception

1

La planification s'effondre alors que les requêtes sont simples

Vérifier le nombre de chunks, et le temps de planification

Prévenir alerter au-delà d'un nombre de chunks par hypertable

2

Un agrégat affiche des valeurs figées depuis plusieurs jours

Vérifier `last_successful_finish` du job de rafraîchissement

Prévenir alerter sur l'ancienneté du dernier succès, pas sur l'échec

3

La compression ne se déclenche pas, le volume grossit

Vérifier l'état du job de bascule, et le nombre de chunks non compressés au-delà du délai

Prévenir alerter sur les chunks éligibles non basculés

Ce qui vient de l'exploitation

4

L'ingestion ralentit progressivement depuis des semaines

Vérifier la taille du chunk actif face à la mémoire, et les événements d'attente

Prévenir suivre le débit d'ingestion comme une métrique, pas comme une impression

5

Une requête est soudainement lente, sans changement applicatif

Vérifier le plan actuel comparé à l'instantané de référence

Prévenir capturer un instantané des statistiques de requêtes avant chaque déploiement

6

L'espace disque ne se libère pas malgré la rétention

Vérifier les chunks entièrement antérieurs à la limite, et l'état du job de rétention

Prévenir alerter sur l'écart entre la fenêtre attendue et la fenêtre réelle

La panne qui vient de la mise en service

Aucun travail d'arrière-plan ne s'exécute. Aucune erreur, aucun ralentissement, aucun message. Un silence complet.

La cause

Le plafond global de processus de travail ne couvre pas les workers de l'extension et le parallélisme. Réglage posé — ou omis — à la mise en service.

Pourquoi elle est la première du catalogue

Elle a été annoncée en M02, rappelée en M11, et elle reste la cause la plus fréquente d'un système qui semble sain.

La vérification

Comparer les deux paramètres, puis compter les processus effectivement actifs. Deux lignes de SQL.

La prévention

Vérifier la présence des workers dans le tableau de bord, et alerter sur leur absence — pas sur leur erreur.



BLOC 15.3

Métriques, alertes, non-régression

Quatre alertes défendables valent mieux que quarante qu'on finit par ignorer.

Huit métriques à exposer

Débit d'ingestion	points par seconde, en moyenne glissante
Ancienneté du dernier point	par flux, détecte un collecteur muet
Retard de rafraîchissement	par agrégat, en écart au présent
Nombre de chunks	par hypertable
Ratio de compression	global, et part de chunks non basculés
Échecs de jobs	et surtout ancienneté du dernier succès
Espace disque	occupé et disponible, avec la tendance
Taux de succès du cache	sur les lectures

Les trois premières sont des signaux précurseurs : elles bougent avant que quiconque ne se plaigne. Les cinq suivantes constatent.

Les quatre alertes du premier jour

1

Aucun worker actif

Le système paraît sain et ne fait plus rien. Panne n° 7.

2

Un job sans succès depuis N fois sa période

Détecte l'échec répété et la suspension oubliée. À limiter aux jobs déjà échus : une politique jamais exécutée affiche `last_successful_finish = -infinity`, faux positif garanti.

3

Un flux sans point depuis plus de N minutes

Détecte le collecteur muet, avant que la courbe ne s'en aperçoive.

4

Espace disponible sous un seuil, avec tendance

La seule qui doive réveiller quelqu'un la nuit.

Aucune de ces quatre ne porte sur une durée de requête. C'est délibéré : une requête lente se voit, un silence non.

Quarante alertes valent zéro alerte

Le seuil absolu

« Alerter si la requête dépasse deux secondes. » Il se déclenche dès qu'une charge inhabituelle passe, et personne ne le regarde plus au bout d'un mois.

Le seuil relatif

« Alerter si le retard de rafraîchissement dépasse trois fois la période du job. » Il s'adapte à la configuration et reste vrai après une modification.

Le test à faire passer à toute alerte avant de l'activer

- Elle se déclenche sur une situation provoquée volontairement
- Elle ne se déclenche pas pendant une semaine de régime normal
- Celui qui la reçoit sait quoi faire, et cette action est écrite
- Elle a un propriétaire nommé, qui peut décider de la retirer

Non-régression de performance

Toute la formation repose sur des mesures. Rien n'a encore été prévu pour qu'elles tiennent après votre départ.

1

Capturer un instantané avant chaque déploiement

Statistiques de requêtes, remises à zéro puis relevées après une période représentative.

2

Rejouer après

Comparer les mêmes requêtes : temps total, nombre d'appels, blocs lus.

3

Comparer les plans des requêtes critiques

Celles de M06. Un plan qui change après une montée de version est un incident silencieux.

4

Consigner l'écart, même quand il est favorable

Un gain inexpliqué est aussi suspect qu'une régression.

C'est le prolongement direct de mesures.md : le fichier ouvert en M02 devient le premier document d'exploitation.



ATELIERS L14 ET L15

Deux pannes, quatre alertes

Diagnostiquer par une requête, puis instrumenter pour ne plus avoir à le faire.

Pannes injectées

SOCLE • pour tous

- Deux incidents ont été introduits à votre insu dans l'instance
- Diagnostiquer en binômes, en appliquant la méthode en cinq temps
- Pour chacun : identifier la cause racine ET la requête qui l'a révélée
- Corriger, et mesurer l'effet de la correction
- Restituer la démarche à la salle, pas seulement le résultat

EXTENSION • pour aller plus loin

- Un troisième incident, non documenté dans le catalogue des sept pannes
- Pour chacun des trois : écrire la requête de prévention qui l'aurait détecté trois jours plus tôt
- Transformer ces trois requêtes en alertes, et les tester

Réussi si les deux causes racines sont identifiées avec la requête de diagnostic qui les a révélées, pas par tâtonnement.

Tableau de bord et alertes

SOCLE · pour tous

- Partir du tableau de bord fourni, déjà branché sur les huit métriques
- Définir les quatre alertes du bloc 15.3, avec des seuils relatifs
- Les déclencher sur situation provoquée
- Vérifier qu'aucune ne se déclenche en régime normal

EXTENSION · pour aller plus loin

- Capturer un instantané de statistiques, déployer une modification, identifier la régression
- Défendre par écrit la liste courte d'alertes du premier jour, en justifiant chaque exclusion

Réussi si les quatre alertes se déclenchent sur situation provoquée, et aucune ne se déclenche en régime normal. Le second critère est le plus difficile.

Pièges et indices

La panne est trouvée, mais par tâtonnement

- Le critère porte sur la requête qui l'a révélée. Reprendre la méthode en cinq temps et refaire le chemin, même après avoir trouvé.

Les événements d'attente ne montrent rien

- Une prise unique attrape mal un phénomène intermittent. Boucler la requête et agréger, plutôt que de la lancer une fois.

Une alerte se déclenche en régime normal

- Son seuil est absolu, ou trop serré. Le second critère de L15 est le plus difficile, et c'est celui qui distingue une alerte utile d'un bruit.

L'instantané de statistiques est vide

- Le module de suivi des requêtes doit être actif avant l'incident, pas pendant. C'est une ligne à ajouter à la mise en service de M02.

État attendu

Livrable du module

- Le jeu de requêtes de diagnostic, rassemblé
- Quatre alertes testées dans les deux sens
- Les requêtes de prévention des deux pannes

Pathologies de M01

6 / 6

Les six symptômes de l'amorce du premier jour sont traités. La dette ouverte en M01 est soldée.

Avant la clôture

- Le jeu de diagnostic et les quatre alertes sont les deux livrables qui servent dès le premier jour de production
- mesures.md, ouvert en M02, devient le premier document d'exploitation
- La séquence de clôture reprend la carte des concepts et les six pathologies, puis chacun écrit son plan d'application



SÉQUENCE DE CLÔTURE

Ce que vous emportez

Formation TimescaleDB · Maîtriser les données temporelles sur PostgreSQL

12 notions, et où chacune a été vécue

Série et point

M03

Mesure ou événement

M03

Cardinalité

M09

Hypertable

M03

Chunk

M04

Dimension

M04

Exclusion de chunks

M04

Hypercore

M09

Seau temporel

M06

Agrégat continu

M08

Politique et job

M10

Agrégabilité

M07

Aucune n'a été seulement énoncée. Chacune a été instanciée sur MISTRAL, et le module indiqué est celui où elle a cessé d'être un mot.

Les six pathologies

Symptôme du premier jour	Résolu par	Module
● Le débit d'insertion s'effondre	chunk dimensionné, COPY	M04 · M05
● La purge bloque la production	suppression de chunk	M10
● Les agrégations sont recalculées	matérialisation incrémentale	M08
● Les données froides coûtent le prix des chaudes	columnstore, rétention	M09 · M10
● Les moyennes sont fausses	pondération par le temps	M07
● Référentiel et mesures vivent séparés	migration sur la cible	M12

Six sur six. L'écart affiché à l'amorce du premier jour a été remboursé, et chaque remboursement a été mesuré.

Ce que la formation ne résout pas

● La cardinalité extrême reste hors périmètre

Plusieurs millions de séries actives disqualifient l'approche. C'était vrai au slide 1.2 du premier jour, et rien depuis ne l'a contredit.

● Le stockage étagé n'existe qu'en service managé

En auto-hébergé, il faut le construire — c'est un projet, pas un réglage.

● L'orchestration entre systèmes reste dehors

Un job ne rattrape pas ce qu'il a manqué pendant une indisponibilité, et n'orchestre rien au-delà de sa base.

● L'effacement n'atteint pas les sauvegardes

Seule leur rétention borne le délai. C'est un fait, pas un défaut de configuration.

● Les durées mesurées ici ne sont pas transposables

Les rapports le sont, les procédures aussi. Les valeurs absolues, non.

Ce qui sert dès lundi

Utilisables tels quels

- checklist-bascule.md, éprouvée en M12
- Le jeu de requêtes de diagnostic de M15
- Les quatre alertes et leur tableau de bord
- La checklist de vérification post-restauration

À transposer sur votre contexte

- mesures.md → la méthode, pas les valeurs
- schema.sql et ses six décisions justifiées
- politiques.sql, valeurs d'atelier et cible séparées
- Le carnet de volumétrie à trois ans

Le fichier le plus sous-estimé mesures.md, ouvert au deuxième module. Il devient le premier document d'exploitation, et la référence de toute non-régression future.

EXERCICE

Votre plan d'application

Trois actions sur votre propre contexte. Chacune avec une date, et un critère de réussite vérifiable.

Action

ce que vous ferez, formulé à l'infinitif

Échéance

une date, pas « rapidement »

Critère de réussite

ce qui sera vrai, et mesurable, quand ce sera fait

Obstacle prévu

ce qui pourrait l'empêcher, et à qui il faut en parler

Une consigne, et une seule la première action doit pouvoir démarrer sans autorisation de personne. Les deux autres peuvent en demander une.

Si vous ne deviez retenir que trois choses

1

Sur des mesures, pas sur du dogme

Chaque décision de ces cinq jours a été tranchée par un chiffre : l'intervalle de chunk, la stratégie d'ingestion, la segmentation, le décalage de rafraîchissement. Aucune ne l'a été par une règle empirique.

2

Le silence est le mode de défaillance

Un job suspendu, un agrégat figé, un worker absent, un seau matérialisé incomplet : rien de tout cela ne produit d'erreur. Ce sont ces situations-là qu'il faut instrumenter, pas les requêtes lentes.

3

La procédure vaut plus que l'opération

La checklist de bascule vaut plus que la migration qu'elle a servi à faire. La démarche de diagnostic vaut plus que la panne qu'elle a résolue.

Ressources et veille

Se tenir à jour

- Les notes de publication, à chaque version mineure
- La matrice de support : elle décide de vos chemins de montée
- Vérifier la version chargée, pas la version installée

Les trois choses à faire ce mois-ci

- Exécuter une restauration, en chronométrant
- Activer le suivi des requêtes, avant d'en avoir besoin
- Poser les quatre alertes, et vérifier leur silence

Le jeu de données MISTRAL

Les signaux sont synthétiques, mais calibrés sur des données SCADA réelles de parcs éoliens publiées en licence ouverte par Cubico Sustainable Investments : courbe de puissance, taux de disponibilité, structure des interruptions. Les références figurent dans le dépôt.